Can Structured State Unify Agentic Execution?

Mehrdad Zakershahrak[1] Samira Ghodratnama[1] Mohammadali Karfarma

[1]School of Computing, Macquarie University, Sydney, Australia

{mehrdad.zakershahrak, samira.ghodratnama}@mq.edu.au

makarfarma@gmail.com

September 2026

Abstract

Context management, coordination, recovery, and verification are often treated as separate problems in agentic systems, although each depends on goals, evidence, authority, effects, and unresolved work. We ask whether a common representation of these distinctions can unify agentic execution. We introduce the *typed execution case*, a persistent, versioned state model with explicit execution relationships, and characterize nineteen structural operation families over it. A common contract framework distinguishes information preservation from authorized state change and yields conditional properties for dependency-closed views, authority non-amplification, and composition. An executable reference model provides 55 passing contract tests, including 6 composed scenarios. A controlled study spanning all nineteen families and seven operation pairs records 3,168 executions satisfying their specified outcome checks. In stress workloads, compaction, projection, and retrieval reduce delivered context by 86.3%, 88.0%, and 77.7%, respectively; caching reduces compute calls by 72.5%, and selective invalidation and merge reduce their respective computation and worker calls by 45.0%. Authorization and completion checks retain correct decisions in all tested cases. Gains vary by workload, and some operations incur additional cost. A separate storage study recovers correctly across 192 trials, including 144 abrupt interruptions. These results establish benefits of individual operations and preservation properties within the tested implementations. A pilot using a model on four tasks yields no fully successful tasks in either arm, leaving improvements in completion by interactive agents and advantages over matched alternative state representations open. The framework thus provides a common, executable basis for studying both the benefits and the limits of structural operations.

Keywords: agentic systems; execution state; typed representations; context management; coordination; recovery; verification

1 Introduction

Language model agents organize work through repeated interactions with tools, environments, and other agents. Reasoning and action loops, together with conversational coordination, provide flexible mechanisms for selecting and distributing that work [30, 31]. Their continuity, however, depends on more than the sequence of messages they produce. An execution may outlive a context window, resume in another process, acquire new evidence, or transfer responsibility to a different worker. At each boundary, the system must determine which parts of the preceding interaction still govern what may happen next.

Consider an agent authorized to place an order. The service commits the purchase, but its acknowledgement is lost. Before the agent can investigate, its context is compacted and execution transfers to another worker. A summary stating that the purchase was attempted leaves several questions open: which proposal was approved, whether the approval remains current, whether the request produced an effect, and whether retrying would duplicate the purchase. The replacement worker needs these distinctions even if the original conversation has been retained in full. The problem is therefore not simply how much history survives, but whether the operational meaning of that history survives a change in representation, process, or owner.

The same problem recurs across apparently different capabilities. Compaction must retain unresolved obligations. Delegation must identify the evidence and authority available to a child task. Merge must determine whether a result remains compatible with its parent state. Recovery must distinguish an absent acknowledgement from an absent effect. Verification must relate the requested outcome to current evidence and unfinished work. These operations share a representational requirement: the relationships governing execution must remain addressable independently of the narrative in which they were first expressed.

We propose the *typed execution case* as a common state model for these operations. A case represents an execution instance through separately identified goals, evidence, authority, proposals, attempts, receipts, effects, and obligations. Versioned relationships express, for example, which proposal a grant authorizes, which attempt a receipt describes, and which source revision supports a decision. Model context becomes a derived view of this maintained state. A change to the view need not change the authority of an action, and a change to the responsible worker need not erase the obligations of the execution.

Our central hypothesis is that *a common typed state reduces semantic loss and repeated reconstruction at the interfaces between agent capabilities.* The proposed unification is representational and operational: capabilities act on a shared account of ongoing work, while retaining distinct algorithms and enforcement mechanisms. It does not require a single planner, a fixed multiagent topology, or deterministic language generation. A language model, symbolic controller, or human operator may propose the next action; the execution case specifies the state against which that proposal is interpreted.

This approach develops the architecture introduced in our companion systems survey around a persistent execution case [35]. Section 2 situates it among explicit state, context management, and transactional execution systems; our focus is the common relationships needed for their operations to compose.

The paper makes four contributions. First, it defines a versioned case model that separates historical records, maintained execution state, context visible to the model, and the external environment. Second, it characterizes nineteen structural operation families and identifies their common state requirements. Third, it develops a contract framework and conditional composition properties, including counterexamples in which individually useful operations conflict. Fourth, it provides executable reference semantics and a controlled empirical evaluation across the operation set, reporting reductions in context volume and repeated work, preserved correctness, effects that depend on composition, and implementation overheads.

The empirical comparisons distinguish enabling a strategy within a typed runtime from replacing the representation itself. The controlled workloads establish benefits of individual operations; an additional recovery study exercises process interruption, while a small pilot using a language model does not establish improved task completion. Section 2 reviews prior work. Sections 3–5 develop the model and its operations. Section 6 defines the broader comparative program. Sections 7 and 8 present the implementation and completed studies; Section 9 discusses their implications and limits.

2 Related Work

2.1 Explicit state and execution based on contracts

Explicit state is a foundation of classical agent architectures [24, 29]. STRIPS and PDDL represent action preconditions and effects [4, 20]; BDI separates informational and motivational commitments [23]; and POMDPs distinguish an environment from the information available to a decision maker [14]. Hierarchical task networks and options further characterize temporally extended work through initiation and termination conditions [3, 25]. These distinctions remain relevant when language models supply action proposals.

Recent execution systems develop related controls for agents based on LLMs. ToolGate grounds tool invocation and state evolution in formal contracts [18], while Atomix studies effect settlement under failures and concurrency [21]. Their guarantees depend on particular execution mechanisms and environmental assumptions. The typed case provides a representation through which those assumptions, permissions, attempts, and observed effects can also participate in context management, delegation, and completion checking.

PROV-DM formalizes derivation and responsibility among entities, activities, and agents [22]. Our companion survey develops an architecture based on a persistent case connecting evidence, proposals, authority, receipts, effects, and responsibility [35]. The present work extends that architecture into an operation set and composition framework. Its focus is the applicability of versioned execution relationships across the lifecycle, rather than provenance or action validity in isolation.

2.2 Context retention and view construction

Knowledge Triage assigns typed knowledge to retention policies for compaction, decomposition, and retrieval [36]. Context Codec separates extraction, normalization, semantic representation, rendering, and preservation assessment [27]. These approaches address the preservation of information whose significance is not captured by its surface length. In the execution case, preservation additionally concerns the applicability of a grant, the uncertainty of an attempt, the base of a delegated result, and the discharge conditions of an obligation.

Personalized summarization provides a related perspective on selective information presentation. Adaptive Summaries [9] and SumRecom [8] use user feedback to select content according to individual preferences, while Intelligent Narrative Summaries [10] supports hierarchical exploration of information and access to additional detail on demand. These approaches motivate distinguishing retained information from the view presented to a particular consumer. The present work applies that distinction to execution state: a decision-specific view must account for current authority, versioned dependencies, uncertain attempts, and unresolved obligations in addition to relevance. Personalized content selection and execution-preserving view construction therefore address related but different requirements.

Related work also examines efficient, personalized information retrieval with LLMs [7] and the role of personalization and human context in LLM applications [6].

ACON optimizes observation and history compression using feedback during development [15]. Headroom provides optimization specific to the content and access to retained original content [12]. Such mechanisms are compatible with the representation family developed here: the case specifies which execution relationships must survive, while a compressor determines how the corresponding information is encoded or externalized.

VCC constructs multiple views of conversation traces linked to their sources and evaluates reflector context in AppWorld [37]. The Google ADK context architecture likewise treats context as a compiled view over richer session, memory, and artifact state [17]. ContextAttest provides a prealpha implementation of typed context records with explicit policy information

and compilation receipts [1]. These systems establish important connections between durable information and selective presentation. The proposed case model develops that connection around live execution state, allowing view transformations to share bindings with authority checks, recovery, merge, and closure.

An architecture for explainable reinforcement learning routes reachability, path, and comparison queries through specialized neural modules over a shared representation of deterministic environments [32]. The execution case applies the same emphasis on explicit queries to recorded execution relationships, including authority and unresolved effects.

2.3 Coordination, persistence, and procedural reuse

AutoGen organizes multiagent interaction through conversation [30], and MultiAgentBench evaluates collaboration and competition under different configurations [38]. The case model is independent of a particular interaction topology. It characterizes the state transferred by delegation and returned by a worker, together with the base versions, resource reservations, and admission conditions that determine how the result can affect the parent execution. Human–AI interaction research also emphasizes preserving human agency and judgment in collaboration with LLMs [33]. In the proposed framework, this concern motivates distinguishing a human instruction or approval from a model-generated proposal and retaining that distinction when responsibility transfers between participants.

Model facilitation and debate over explanations have also been proposed for language model alignment [34]. The case model can retain resulting proposals, explanations, and judgments with their sources and authority status as execution proceeds.

Event sourcing and LangGraph persistence support reconstruction and checkpointed continuation [5, 16]. The typed case specializes persisted state around execution obligations and the distinction between historical decisions and current authority. This permits restoration to compose with reconciliation and verification while retaining the assumptions about external services required for safe continuation.

Voyager acquires executable skills through interaction and feedback [28]; SIGIL compiles skill requirements into typed programs and runtime harnesses [2]. These contributions concern reusable descriptions of possible executions. A case instead represents one evolving execution instance. Linking the two through source evidence, preconditions, effects, and completion obligations provides a basis for extracting candidate procedures while separating reusable behavior from permission specific to the instance.

The composition framework in Section 5 specifies how these mechanisms specific to each capability can consume common execution relationships.

3 A Typed Model of Execution State

3.1 Execution state and its representations

The proposed representation separates four objects that are frequently combined in an agent's working history. Let H_t denote the recorded interaction history, $\mathcal{C}_t$ the execution case, $\mathcal{V}_t^{p,d}$ a view supplied to principal p for decision d, and $\mathcal{E}_t$ the external environment. History records observations and messages; the case maintains admitted execution state; a view selects the state available for a particular decision; and the environment contains the resources whose effects determine task success.

This separation permits different notions of continuity. Retaining H_t preserves an account of prior interaction, whereas restoring $\mathcal{C}_t$ recovers the obligations and commitments governing continuation. Reconstructing $\mathcal{V}_t^{p,d}$ supplies a particular decision maker with relevant state, but

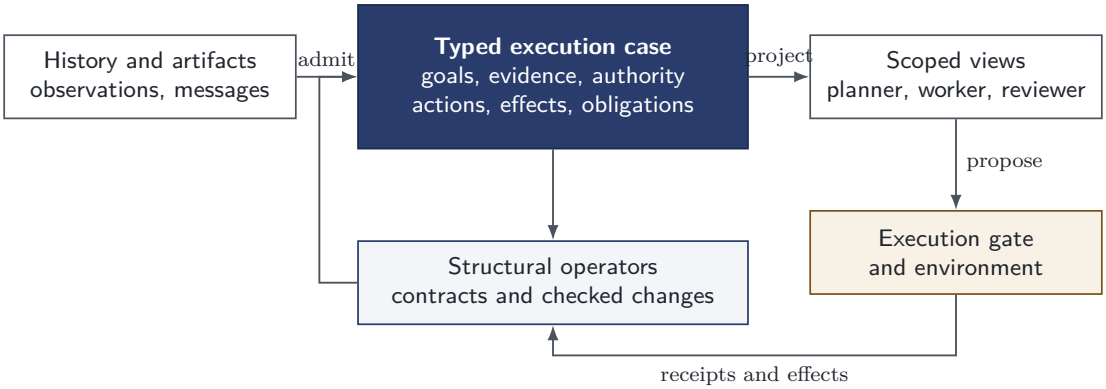


Figure 1: A typed execution case mediates between recorded interactions, views visible to the model, and execution affecting the environment. Structural operators transform the case or its views, while the execution gate evaluates proposed actions against current authority and environmental preconditions.

does not establish that $\mathcal{E}_t$ remains unchanged. Consequently, an execution can retain a correct historical record while requiring new evidence before another action is admissible.

A loop of reasoning and action mediated by the case can be written as

$$\mathcal{C}_{t+1} = \mathrm{Admit}(\mathcal{C}_t, e_{t+1}), \tag{1}$$

$$\mathcal{V}_{t+1}^{p,d} = \mathrm{Render}(\mathrm{Project}(\mathcal{C}_{t+1}, p, d)), \tag{2}$$

$$\widetilde{a}_{t+1} \sim \pi(\cdot \mid \mathcal{V}_{t+1}^{p,d}), \tag{3}$$

$$a_{t+1} = \mathrm{Gate}(\widetilde{a}_{t+1}, \mathcal{C}_{t+1}, \mathcal{E}_{t+1}), \tag{4}$$

$$(o_{t+2}, \mathcal{E}_{t+2}) \sim \mathrm{Execute}(a_{t+1}, \mathcal{E}_{t+1}). \tag{5}$$

Here, admission establishes the status of an incoming event, projection and rendering construct decision context, and π proposes an action. The gate may admit, deny, defer, or escalate that proposal. Execution produces an observation that becomes a candidate for subsequent admission. This formulation retains the flexibility of interaction between reasoning and action [31], while making the transitions between observation, interpretation, authorization, and effect explicit.

3.2 Records and versioned relationships

Definition 3.1 (Typed execution case)**.** *A typed execution case at version v is a tuple*

$$\mathcal{C}_v = \langle N_v, D_v, L_v, B_v, \rho_v \rangle,$$

where N_v is a set of typed records, D_v is a set of relationships sensitive to identity and version, L_v is an ordered record of admitted transitions, B_v contains policy and resource bounds, and ρ_v resolves retained artifacts subject to access and retention rules.

A record has a stable identity, kind, revision, typed body, lifecycle status, provenance, dependency bindings, and disclosure scope. These attributes support two complementary functions. The type identifies the role of a record in execution; its relationships determine the conditions under which it can be used. A grant, for example, is meaningful as authority only in relation to its principal, proposal revision, target, scope, and validity interval. Similarly, a receipt must identify the attempted operation whose outcome it reports.

The distinction between control state and bulk payload follows from these execution roles. An approval's target and expiry belong to control state because changing either can change the set of permitted actions. A lengthy supplier document can be retained behind a reference, provided the decisions that depend on it preserve the necessary source and revision bindings. Externalization

Record family	Required distinction	Representative binding
Goal	Requested outcome versus current progress	Acceptance predicate and accountable owner
Evidence	Observation, inference, assumption, and conflict	Source identity, revision, time, and derivation
Authority	Permission versus model intention	Principal, proposal revision, scope, and validity
Proposal/attempt	Intended operation versus dispatched attempt	Logical operation ID, concrete target, and attempt ID
Receipt/effect	Returned response versus established outcome	Attempt identity and authoritative effect observation
Obligation	What remains to establish or repair	Goal, required evidence, owner, and discharge condition
Coordination	Delegated work versus parent authority	Base version, read/write set, reservation, and admission rule
Lifecycle and resources	Available work versus expired or exhausted work	Retention, invalidation, consumption, and termination reason

Table 1: Semantic distinctions and representative bindings in a typed execution case. The record families specify execution roles rather than a mandatory storage layout.

therefore changes where information is represented without necessarily changing its operational significance.

Versioning further separates historical validity from current applicability. Evidence may accurately describe an earlier state while no longer supporting a proposed action. A child result may be well formed yet incompatible with the parent's revised constraints. General provenance models such as PROV-DM provide distinctions among entities, activities, derivations, and responsible agents [22]. The execution case associates such relationships with the validity conditions of ongoing work.

3.3 State validity and lifecycle

The case records not only what an object represents but also its current disposition. An obligation may be open, blocked, discharged, cancelled by an authorized decision, or superseded by a successor. An attempt may be prepared, dispatched, uncertain, confirmed, failed, or awaiting compensation. Authority can expire or be revoked without deleting the historical grant. A delegated branch may return a result before that result is admitted into the parent case. Refinements specific to the domain are possible, but each transition requires a defined interpretation.

Uncertainty is a persistent disposition within this lifecycle. In the order example, a lost acknowledgement establishes neither failure nor success. Reconciliation may confirm an effect, establish that it did not occur, or leave the outcome unresolved. The corresponding obligation remains open until adequate evidence or an authorized alternative disposition is available. This treatment is consistent with the distinction between an environment and an agent's informational account of it [14].

Figure 2 illustrates these distinctions through the purchase example. After the service commits the purchase but its acknowledgement is lost, the case retains an uncertain attempt and an open obligation to establish the outcome. Reconciliation admits authoritative evidence linked to the same logical operation, allowing the attempt to be confirmed and the obligation to be discharged. The external effect exists in both panels; what changes is the execution state's evidence about that effect.

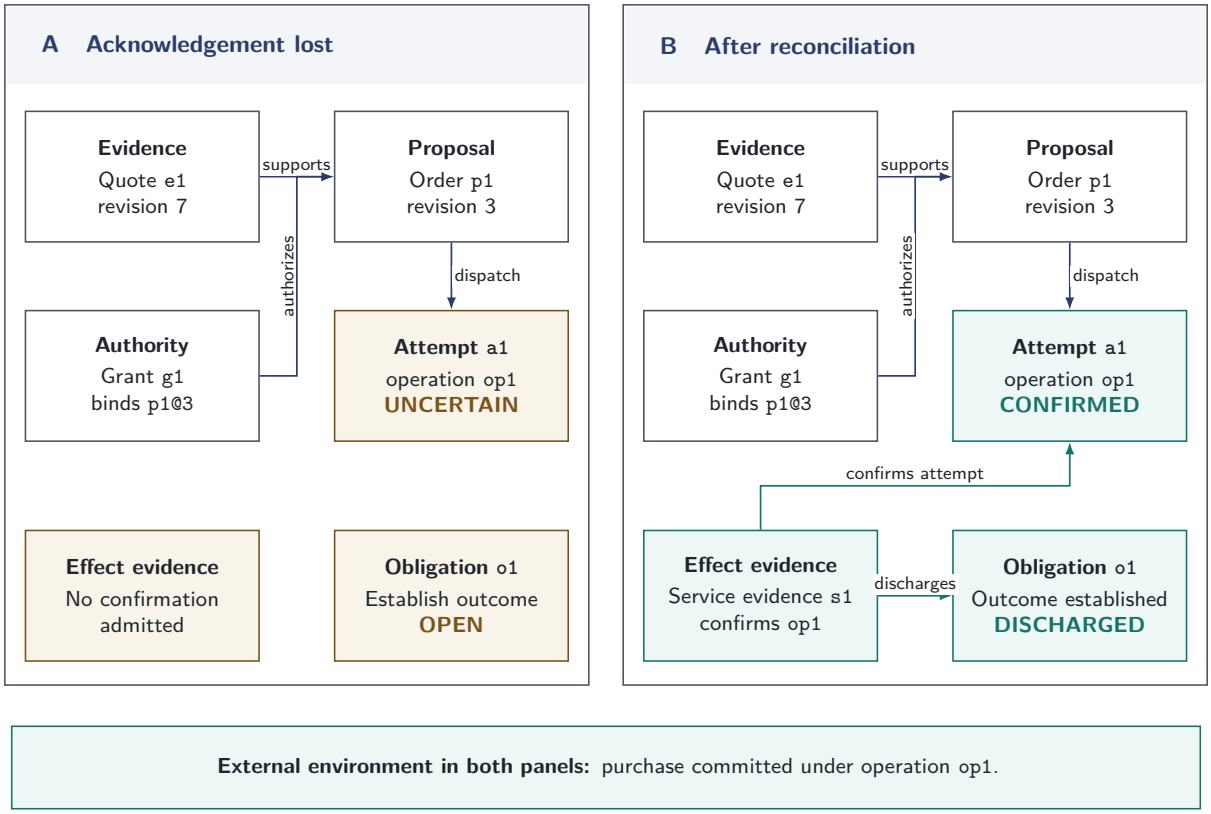


Figure 2: A typed execution case across acknowledgement loss and reconciliation. (A) The purchase has committed externally, but no confirmation has been admitted; the attempt remains uncertain and the outcome obligation remains open. (B) Service evidence confirms the existing operation and discharges the obligation without repeating the purchase. Record identifiers and revisions are illustrative.

Three forms of validity follow. *Structural validity* concerns schema and reference integrity. *Decision validity* concerns the current applicability of evidence, dependencies, and authority. *Outcome validity* concerns whether external evidence establishes the requested effect. These distinctions are consequential for composition: a structurally valid restored case may contain an expired grant, and an authorized action may still fail to produce its intended outcome.

3.4 State construction and provenance

A case can be maintained incrementally as execution events occur. Dispatch supplies an attempt identifier, an authorization service supplies a grant reference, and an evaluator records an observed postcondition. Natural language interpretation remains necessary for many goals and observations, but its outputs retain their relationship to the underlying inputs. An inferred requirement is thus distinguishable from an explicit instruction, and a reported effect from an independently checked postcondition.

Retrospective transcript extraction is a separate construction path. An extractor may identify a goal or commitment in a sentence, but it cannot recover an unrecorded permission or a missing service receipt. Such cases require source spans, extraction status, and unresolved fields. The distinction between native admission and retrospective extraction also determines the evaluation design: the quality of the representation and the accuracy of its construction are separate experimental factors.

3.5 The structural operation interface

A structural operator consumes a case or view, explicit arguments, a principal, and the relevant policy and environment assumptions. Its output is a case delta, view, derived artifact, or verdict, accompanied by an operation record identifying the inputs, outputs, versions, affected identities, and disposition. Denial, conflict, infeasibility, stale input, and unresolved effect are defined outcomes alongside success.

The shared interface accommodates different semantics. Compaction changes a representation while preserving specified information; projection restricts an observation; revocation changes authority; merge admits new state; and replay reconstructs an earlier state. Their commonality lies in the objects and relationships on which they operate, rather than in an obligation to preserve every field. The next section develops the resulting operation families, and Section 5 characterizes their preservation and transition contracts.

4 Structural Operations over Execution State

A shared execution representation is useful to the extent that it supports operations whose meaning extends across individual model invocations. The nineteen operation families developed below are organized around six capabilities: context management, governance, recovery, coordination, verification, and reuse across the state lifecycle. Within each family, the defining question is which execution relationships must be preserved or deliberately changed. Appendix A provides the corresponding implementation and test index.

4.1 Context management and state transformation

Canonicalization and admission. Canonicalization gives heterogeneous inputs a representation on which identity, comparison, and dependency tracking can operate consistently. It establishes supported value types, stable identities, revisions, and explicit references. Admission then determines whether a normalized record may participate in authoritative execution state. Separating these operations allows a correctly parsed source to remain untrusted and an inferred claim to remain distinguishable from an observation.

This distinction also determines what counts as equivalent. Two encodings of the same record with integer values may share a canonical digest, whereas identical supplier statements from different sources retain separate provenance. Normalization must therefore preserve record kinds and source relationships, reject unsupported coercions, and validate dependency bindings. The resulting benefit is a stable basis for subsequent operations, rather than repeated interpretation of surface text.

Compaction. The case model permits compaction to distinguish control state from bulk evidence and optional narrative. Exact identifiers, current grants, unresolved attempts, and discharge conditions can be retained while document payloads are externalized, repeated content is factored, or material is summarized under an explicit loss policy. Typed retention and representations that preserve commitments provide related approaches to selective preservation [27, 36]; Headroom supplies mechanisms for compression adapted to the content and retrieval of original content [12].

Compaction is consequently defined relative to the execution queries that must remain answerable. In the procurement example, removing a quote from routine context is compatible with retaining its accepted terms, source revision, approved proposal, and open purchase obligation. A reference satisfies that requirement only while its resolver remains available and authorized. If the required state cannot fit within the declared budget, infeasibility is the appropriate result. Context reduction is a potential efficiency benefit whose net value depends on construction, retrieval, and subsequent use.

Projection. Projection constructs a view specific to a decision from a case. A planner may require alternatives and constraints, an executor a concrete proposal and its grant, and a reviewer effect evidence and remaining obligations. These views differ because their decisions differ, even when they refer to the same execution. Selection therefore depends on both relevance and the dependencies required to interpret the selected records.

A pricing worker, for example, can receive supplier evidence without the capability to commit a payment. If a required dependency lies outside the worker's disclosure scope, the projection must expose insufficiency, deny the view, or use an authorized abstraction. Minimality is relative to the declared roots and dependency relation: it identifies a least sufficient closure for specified queries, rather than an optimal prompt for every possible model response. Section 5 formalizes this distinction.

Retrieval. Retrieval locates an identified record; rehydration restores its referenced detail into an authorized view. References sensitive to versions distinguish the evidence used for an earlier decision from a newer document at the same address. This supports a form of continuity in which detail can leave the model's context without losing its relationship to the decision it informed.

The resolver must distinguish a retained original, a superseded revision, denied access, and unavailable content. Reconstructing the shipping clause used in a purchase decision requires the requested source version, not merely a plausible clause from a current document. Integrity and access checks therefore accompany rehydration. Access on demand can reduce routine context volume, although its behavioral value also depends on whether the controller requests the relevant detail when needed.

Deduplication and caching. Stable identities and dependency bindings allow repeated payloads and computations to be recognized without collapsing their execution roles. Several workers can share a retained document or reuse a validated extraction while preserving separate observations and responsibilities. Memoization extends this principle to computed results, whose reuse depends on the source versions, parameters, policy, and tool semantics under which they were produced.

The critical boundary is between reusing information and repeating an effect. A cached response does not establish a new service effect, and two requests with different intended effects remain separate logical operations even when their text is similar. Likewise, a changed quote or recipient policy can invalidate an otherwise identical cached view. The case connects reuse to explicit equivalence and freshness conditions, enabling reduced recomputation where those conditions continue to hold.

Invalidation and recomputation. A structural difference identifies changed records or fields. Dependency bindings propagate that change to affected decisions, derived artifacts, and delegated tasks. Invalidation alters current applicability without necessarily removing the historical explanation: an earlier ranking may remain available for audit while no longer supporting execution.

For example, a revised delivery date can invalidate a supplier ranking and its proposed order while leaving unrelated tax evidence current. Selective recomputation then operates on the affected dependency closure. Its precision depends on the granularity and completeness of the recorded relationships. Versioning at the record level can invalidate more than necessary, whereas omitted dependencies can leave an obsolete conclusion admissible. The representation makes this tradeoff explicit and gives recomputed results new evidence bindings.

4.2 Authority, disclosure, and resource control

Authorization and revocation. Authorization relates a principal to a concrete operation under a particular policy and task state. A grant binds the proposal revision, target, parameters, scope, and validity interval; the existence of a well formed proposal does not imply permission to execute it. Delegated authority is similarly bounded by the parent grant rather than inferred from a role description.

This binding gives context transformations a precise authority boundary. Approval for one order cannot be reused for a revised price simply because the approval sentence remains in context. Dispatch rechecks the current grant and preconditions outside the model's narrative, while revocation changes the set of admissible actions without erasing the historical decision. Systems grounded in contracts such as ToolGate provide a related execution mechanism [18]. The shared case exposes the state required by such enforcement to other operations, including projection, invalidation, and recovery.

Scoped disclosure. Projection addresses informational need; disclosure control addresses permission to reveal information. Their intersection defines an admissible view. A logistics worker may require a delivery obligation and address while being excluded from payment credentials or confidential supplier notes.

Disclosure scope applies to dependencies and derived artifacts as well as original fields. A summary, identifier, trace label, or omission pattern can reveal information removed from the visible source. A redaction contract must therefore specify observable channels and the renderer's trust boundary. Where a task cannot be supported without inaccessible evidence, that conflict becomes an explicit outcome. The case supplies the relationships needed to enforce and audit scoped disclosure; broader information flow guarantees depend on the surrounding system.

Resource control and termination. Resource records extend the same treatment to time, actions, tokens, expenditure, or other controlled units. Reservations assign part of a parent budget to child work before consumption occurs. Settlement records actual use and releases the remainder, preventing the parent and its children from independently spending the same allowance.

Monitoring connects these records to progress and termination. Repeated proposals, unresolved obligations, and exhausted budgets have different interpretations and can trigger different responses. A stalled worker may be stopped and its obligations reassigned, whereas verified completion discharges the relevant work. This distinction supports bounded execution and attributable cost without equating termination with success. Enforcement must occur at interfaces that control the resource being accounted for.

4.3 Recovery and effect reconciliation

Checkpointing. A checkpoint records a case version together with the transition information needed to reconstruct its continuation. Restoring that checkpoint recovers ownership, reservations, evidence bindings, and unresolved obligations, in addition to task content. Event sourcing and durable workflow systems supply established mechanisms for reconstruction of recorded state [5, 16].

The restored state remains subject to its schema, reducer version, and conditions on reference availability. In particular, restoration does not extend an expired grant or establish that an earlier environmental observation is still current. Its contribution is continuity of the internal execution account: a successor can identify an uncertain attempt and the work needed to resolve it without inferring that state from a summary. Durable storage and environment reconciliation provide the complementary mechanisms required for recovery.

Reconciliation and compensation. Reconciliation determines the disposition of an uncertain effect from appropriate evidence. Compensation performs a new operation intended to address an earlier effect. Observing that an order exists resolves uncertainty; cancelling that order changes the environment again. Maintaining separate attempt, effect, and obligation records allows the system to select either operation as required.

A missing acknowledgement therefore remains unresolved until evidence supports a disposition. Logical operation and attempt identities connect the inquiry to the original request, while any compensating action receives its own authority and precondition checks. Transactional work such as Atomix studies effect settlement under faults and concurrency [21]. The case records the evidence and obligations on which those mechanisms depend, preserving the distinction between a confirmed failure and an uncertain result across compaction, restoration, and handoff.

4.4 Delegation, coordination, and handoff

Delegation. Forking creates a child execution from a specified parent state. Delegation assigns a subgoal, readable evidence, permitted actions or writes, resource reservation, and return conditions. Hierarchical task models and temporal abstractions provide foundations for such bounded units of work [3, 25].

A child owns its subproblem, while the parent retains responsibility for admitting the result and determining overall completion. Supplier checks can thus proceed with different views and local obligations without directly rewriting the parent plan. The return artifact identifies its base version, evidence, consumed resources, and unresolved issues. Duplicate delivery remains one result rather than additional progress. This structure supports localized responsibility and failure while making integration costs explicit.

Parallel execution. Scheduling determines which ready tasks can execute under their dependency, capability, and resource constraints. Declared read/write footprints expose one basis for independence: analyses of supplier documents can proceed concurrently, whereas competing updates to one order require coordination. A collection of agents is not itself evidence that their work is independent.

Parallel execution requires sufficiently complete footprints and an explicit policy for unknown conflicts. Conflicting work may be serialized, reserved, or reconciled, and waiting branches may require refreshed views before execution. Shared budgets and external effects introduce additional constraints. These conditions connect scheduling to the same state used by delegation and merge. Potential improvements in makespan or complementary evidence must be evaluated together with coordination overhead, repetition, and cost [30, 38].

Merge. Joining receives child outputs; merging determines which returned changes become part of the parent case. Admission compares a result's claims, artifacts, evidence bindings, observed effects, resource use, and open obligations with its base version and delegated contract. The outcome may be acceptance, rejection, repair, or a represented conflict.

This distinction permits disagreement to remain informative. Conflicting supplier dates can be retained as separate observations pending resolution rather than collapsed into a single fluent answer. Merge validates the proposed change before admission, rejects unauthorized writes, preserves provenance, and settles reservations once. Its scope is the admission of state: external effects already performed by a branch require separate reconciliation or compensation. Stale results similarly require revalidation rather than automatic incorporation.

Handoff and migration. Handoff transfers responsibility for an execution; migration translates its state across schema or runtime boundaries. Both require agreement on meaning, not merely matching field names. A destination that interprets a receipt as completion or

drops an unsupported obligation state can change the execution while preserving its superficial serialization.

A transfer contract therefore covers record kinds, lifecycle states, identity mapping, capability semantics, and reference access. The incoming controller must be able to distinguish established outcomes from pending decisions and to validate authority in its own context. An imported grant records earlier permission; it is not automatically a capability in another runtime. With these conditions satisfied, a compact case can support continuity across agents, models, and human reviewers without requiring them to reconstruct the full interaction history.

4.5 Verification and audit

Replay and audit. Replay reconstructs recorded state under specified event and transition semantics. Tracing follows the relationships between proposals, supporting evidence, gate decisions, attempts, and effects. Audit evaluates that recorded path against its contracts. Together, these operations provide an account of how the execution proceeded and which evidence supported its admitted decisions.

Their object is the recorded execution, rather than unrecorded model reasoning or a fresh execution of the same actions. VCC's trace views linked to sources illustrate the value of multiple inspectable representations [37]; the case connects such views to live authority and lifecycle state. Missing events, unavailable artifacts, or unsupported reducer versions limit reconstruction. Where adversarial integrity is required, an external trust anchor must protect the record against wholesale replacement.

Completion verification. Verification relates the goal's acceptance predicate to effect evidence, collateral constraints, and remaining obligations. Closure records the resulting disposition of the case. Producing an artifact, submitting it, and establishing its acceptance may be distinct steps, even when a natural language summary treats them as one achievement.

An order draft, for example, does not establish that the requested purchase occurred. Current effect evidence must support the relevant outcome, and unresolved effects or required work prevent unqualified closure. Authorized cancellation and partial completion are separate dispositions. Stateful evaluation environments motivate this distinction between intermediate activity and task outcome [19, 26]. By making the dependencies of completion explicit, the case permits the completion decision itself to be inspected and tested.

4.6 Procedural reuse and state lifecycle

Procedure extraction. A completed execution can provide a candidate reusable pattern comprising preconditions, accepted inputs, action dependencies, expected effects, and termination conditions. The derived artifact may be a parameterized procedure, skill, test, or training example. Approaches such as Voyager and SIGIL demonstrate relevant forms of executable skill reuse and typed compilation [2, 28].

The execution case contributes an evidentiary basis for derivation: the procedure can retain the source decisions, checked effects, and discharge conditions that motivated it. Reuse nevertheless separates the pattern from identity and authority specific to the instance. A template for an order update requires a fresh target version and a new grant when instantiated. Transfer evaluation must determine whether the pattern remains applicable beyond its source case; a verified instance supplies evidence for a candidate procedure, not a universal policy.

Retention and deletion. Retirement removes an item from active use, expiry ends a validity interval, and deletion removes retained content within a specified management boundary. These operations affect more than the original record. A case's dependencies can identify derived

caches, views, procedures, and reconstruction paths whose continued use relies on the retired material.

For instance, deleting a supplier document can invalidate a cached extraction and restrict subsequent replay. A retention contract specifies which original and derived copies are covered, where deletion is enforced, and which historical relationships may remain. Tombstones can represent unavailability without retaining prohibited payload. Independently exported information lies outside a local store's deletion boundary. The interaction between retention and reconstruction is therefore a composition problem, not merely a storage policy.

Continuity across operations. The families above reuse execution relationships while imposing different preservation and transition requirements. Section 5 specifies when those requirements are compatible and where one operation can invalidate the assumptions of another.

5 Operation Semantics and Composition

5.1 Preservation and state change

Unifying operations over a common state requires a distinction between their shared interfaces and their different effects. We identify five contract classes: preservation, scoped observation, authorized transition, reconstruction, and derivation. Preservation specifies which execution queries retain their answers. Scoped observation additionally constrains what a recipient may observe. Authorized transition relates preconditions to permitted state changes. Reconstruction identifies a target historical state and transition semantics, while derivation binds an artifact to its source and subsequent conditions of use.

Let Q be a finite set of execution queries, such as the identity of an outstanding attempt, the revision to which a grant applies, or the set of unresolved obligations for a goal. Write $I_Q(\mathcal{C})$ for their interpretation under declared version, policy, and assumptions about reference resolution. For a preservation operator T, the intended property is

$$I_Q(T(\mathcal{C})) = I_Q(\mathcal{C}). \tag{6}$$

Equation 6 defines equivalence with respect to Q, rather than all possible future model responses. Operators such as revocation, deletion, and closure intentionally change relevant state and instead require a relation $R_T(\mathcal{C}_v, \mathcal{C}_w)$ specifying permitted changes and retained invariants. The distinction allows transformations that preserve information and operations that change state to participate in the same execution without assigning them incompatible correctness criteria.

Reference resolution is part of the interpretation function. A query concerning an unresolved effect cannot be preserved by replacing its answer with an indistinguishable absence. Similarly, externalizing evidence preserves a required query only under the stipulated availability and access conditions of the resolver. Unavailable, denied, stale, and unresolved outcomes must therefore remain semantically distinct.

5.2 Dependency-closed views

Let D be the finite dependency relation among admitted records. For a root set R, define

$$\mathrm{cl}_D(R) = \mu X\big(R \cup \{y : \exists x \in X, (x, y) \in D\}\big),$$

the least fixed point containing the roots and their dependencies. The relation is specific to the decision: a provenance dependency may be needed for audit but not for a narrowly authorized action whose admissibility has already been certified under a separate contract.

Proposition 5.1 (Least dependency-closed selection)**.** *For finite D, repeated dependency expansion terminates and returns the unique least dependency-closed superset of R. If every member is*

current and authorized for the recipient, exact rendering of that set preserves the record queries defined over the selected closure.

Proof. Expansion is monotone over the finite set of record identities, so it terminates. Any dependency-closed superset of R contains each element added at every expansion step, by induction. The terminal set is therefore contained in every such superset and is itself closed. Exact rendering preserves the values and bindings used by the stipulated queries; currentness and authorization are separate hypotheses of the result. □

Minimality is relative to D and R. The proposition characterizes the records needed to preserve specified dependencies, without optimizing their token encoding or determining which roots a decision should require. If the closure contains inaccessible records, exact preservation and the disclosure constraint cannot both be satisfied by that view. An authorized abstraction may support a different contract; otherwise, the conflict yields an explicit infeasibility result.

5.3 Authority preservation

Assumption 5.2 (External authority gate)**.** *For a concrete proposal a, execution permission is computed by a gate from the current authoritative case, authenticated principal, applicable policy, and required current environment checks. Text in the model context cannot create a grant. A representation transformation cannot mutate those authoritative inputs.*

Proposition 5.3 (No authority amplification by view transformation)**.** *Under the assumption of an external gate, compaction or projection alone cannot turn a denied concrete proposal into an authorized proposal when the authoritative inputs are unchanged.*

Proof. The gate's decision is a function of the stipulated authoritative inputs. The transformation changes none of them, so the decision for the same concrete proposal is unchanged. The model may subsequently propose a different action, but that action is evaluated independently by the same gate. □

The proposition locates authority in the maintained execution state rather than its rendering visible to the model. It distinguishes a gate that resolves a grant against current authoritative inputs from a controller that reconstructs permission from an approval sentence. Restored grants must likewise be reevaluated under current validity and revocation conditions. Representation invariance is thus a property of the boundary between state and execution, not of the linguistic form of a permission statement.

5.4 Sequential composition

Following precondition/postcondition reasoning in program logic [13], let Σ contain the case, current view, derived artifacts, and relevant environment assumptions. Represent each operator as a partial transition on Σ with a defined failure result. This formulation accommodates operations that produce views or change state within the same composition rule.

Proposition 5.4 (Conditional composition)**.** *Suppose successful operator T_1 establishes P_1, operator T_2 requires P_2, and P_1 implies P_2 in the same identity, version, policy, and reference resolution context. If both operators preserve invariant J outside their declared permitted changes, their successful composition preserves J. An intervening change that falsifies P_2 requires revalidation.*

Proof. After T_1, J and P_1 hold by hypothesis. Hence P_2 holds and the successful execution of T_2 preserves J. If the intervening context changes, the implication need no longer apply; the result then makes no claim until the precondition is reestablished. □

Composition	Failure when the boundary is implicit	Required contract condition
Compact → project → retrieve	A referenced obligation or source is no longer accessible.	Preserve required closure and resolver conditions.
Fork → parallelize → merge	Workers overwrite state based on incompatible versions.	Bind bases, footprints, authority, and result admission.
Dispatch → restore → retry	A lost acknowledgement causes duplicate effects.	Preserve attempt identity and reconcile uncertainty.
Revoke → cache reuse	A cached approval survives its validity interval.	Recheck current authority and invalidate dependencies.
Handoff → migrate → verify	The destination treats unsupported states as success.	Validate semantics and preserve open obligations.
Delete → replay	Replay resurrects covered content from an old snapshot.	Apply durable retention decisions to references and derivatives.

Table 2: Composition failures and the execution relationships required to prevent them. Each row identifies a boundary at which a locally valid operation can invalidate the assumptions of its successor.

The case model specializes this rule through the predicates propagated across operator boundaries: open obligations, authority bindings, attempt and receipt identities, resource reservations, available sources, and admissible branch versions. A compaction and handoff chain, for example, requires the transfer preconditions to hold for the compacted representation and its resolver. A revocation between those operations can invalidate that implication even when the serialized records are unchanged.

5.5 Concurrent composition

Let branches i and j read record sets R_i, R_j and propose state writes W_i, W_j. A sufficient independence condition for deterministic computations without effects on a common base is

$$W_i \cap (R_j \cup W_j) = \varnothing, \qquad W_j \cap (R_i \cup W_i) = \varnothing. \tag{7}$$

The sets must include every relevant dependency and conflict resource. Shared budgets require separate reservations; shared external effects require their own coordination.

Proposition 5.5 (Domain result independent of order)**.** *If Equation 7 holds, each branch's proposed values depend only on its declared reads, and admission rechecks the common base and applicable constraints, applying the two admitted deltas in either order produces the same domain values on their union of writes. This does not require identical event order, record numbering, or audit metadata.*

Proof. Neither branch changes the other's read values, so both compute the same proposed values in either order. Their write sets are disjoint, so neither overwrites the other's proposed values. The rechecked constraints and reserved shared resources rule out the additional conflicts excluded by the premises. □

The independence condition depends on complete footprints. Two workers updating different orders may still conflict through a shared value for the remaining budget. Likewise, serializing conflicting jobs does not repair a stale view from which a branch has already computed its result. Scheduling and admission must therefore share base versions, dependencies, and resource constraints; ordering by itself is insufficient.

5.6 Limits of composition

Table 2 identifies representative failures of these conditions. The final case exposes a substantive incompatibility: unrestricted replay of sensitive payload and irreversible deletion of that payload cannot both be guaranteed within the same retention boundary. Deletion must instead induce a revised reconstruction contract specifying which relationships remain inspectable and which content is unavailable. More generally, unification makes such conflicts expressible in a common state model; it does not require every operation pair to commute or preserve the same information.

6 Research Agenda

The central hypothesis gives rise to two distinct empirical questions: what individual structural strategies enable, and whether a common representation improves their composition relative to equally capable alternatives. The completed studies reported later in Section 8 address effects of individual operations, storage recovery, and readiness for execution using a model. This section places those studies within the broader research program and specifies the additional comparisons needed for claims about tasks and representations. All nineteen families remain in scope; the prospective extensions below are distinct from the completed measurements.

6.1 Comparative design

The central hypothesis predicts benefits at the interfaces between capabilities. Testing it requires distinguishing the effect of a common representation from the effects of compression, stronger policy gates, better tool access, or more reliable services. A suitable comparison therefore holds the model, task information, access to raw artifacts, executor, policy enforcement, retry semantics, effect evaluator, and resource limits fixed. The experimental factor is how execution state is represented and propagated across the selected operations.

Three representation controls are particularly informative: a ledger in structured prose, a complete uncompressed case, and a compact or projected case. The first tests whether the explicit case model offers an advantage over a competent textual state representation; the second separates representation effects from compression effects; and the third measures the consequences of selective presentation. A further architectural comparison contrasts composition using shared state with equivalent capabilities connected through separate state interfaces. Table 3 identifies outcomes appropriate to the different operation families.

6.2 Evaluation tasks and measures

AppWorld and ToolSandbox provide candidate environments for interactive tasks with outcome checks sensitive to state [19, 26]. Injected interruption or concurrency faults should be reported as separate experimental conditions rather than combined with official unmodified benchmark scores. Migration, retention, and procedure transfer require dedicated workloads with explicit destination and lifecycle semantics.

6.3 Baselines and study protocol

Context comparisons should include competent implementations of Headroom, ACON, Knowledge Triage, and Context Codec where their interfaces fit the task [12, 15, 27, 36]. VCC and context construction in the style of ADK provide additional reference points for scoped views [17, 37]. Implementations and adaptations must be pinned to specific versions before evaluation. These systems are comparison candidates, not baselines whose reported results have been reproduced in the present study.

Question	Outcomes	Necessary comparison
Context efficiency	Net episode tokens and cost; failures due to missing required context; task completion	Structured prose, practical/semantic compressors, full versus compact/projected case
Governance	Invalid proposals, realized violations, benign rejection, and disclosure failures	Equal execution safeguards and equal admitted information
Recovery	Continuation success, repeated operations, duplicate effects, and recovery cost	Strong checkpointed application state and the same service guarantees
Coordination	Makespan, accepted progress, conflicts, stale work, and resource use	Single agent and multiagent alternatives with explicit state
Audit/closure	Source traceability, false completion, and unresolved work	Independent effect evaluator with measured coverage
Portability and reuse	Semantic fidelity, adaptation cost, and performance on new tasks	Real destination adapters and held-out procedure instances
Composition	Boundary mismatches and repeated reconstruction across chains	Matched capabilities using separate state and integration interfaces

Table 3: Prospective extensions beyond the completed controlled studies. These outcomes and representation controls target claims about tasks and interactions between capabilities not established by the present mechanism measurements.

A confirmatory design should use development tasks for adapter construction and pilot variance estimation, followed by a frozen split by task family and a calculation of sample size appropriate to the primary endpoint. Repeated seeds are clustered within tasks. Reporting includes unconditional cost and completion distributions, invalid runs, truncations, abstentions, retries, exhaustion, and exclusions. Procedure reuse additionally requires new instances and accounts for the cost of constructing and validating the reusable artifact. Such a design permits the benefits of structure to be assessed operation by operation and, crucially, at the boundaries where a shared case is hypothesized to reduce semantic loss.

7 Reference Implementation and Validation

7.1 Runtime architecture

We implement the case model as a Python reference runtime comprising typed records, a versioned case, a payload store, delegated branches, and a simulated external service. The implementation provides an executable instance of every operation family. Its canonical value domain consists of strings, booleans, integers, nulls, lists, and dictionaries with string keys; unsupported values and coercions are rejected. This restricted domain makes serialization and identity behavior directly testable.

Records retain stable kinds and increasing revisions, with dependencies bound to the revisions on which a decision or artifact relies. Currentness is checked transitively. Admitted changes append events linked by hashes, and restoration reconstructs the state from a checkpoint's event sequence before comparing the result with its recorded state. Payloads and cached values reside in a separate managed store, allowing compaction to retain exact control records and content references. Rehydration checks recipient scope and content integrity; deletion tombstones covered payloads and invalidates dependent content.

The execution gate resolves grants against a proposal revision, principal, validity interval, and current target version. The simulated service maintains versioned values and an effect journal indexed by operation. Dispatch records an uncertain attempt before invoking the service, so a lost acknowledgement leaves a persistent reconciliation obligation. Reconciliation consults the

Family	Implemented reference behavior	Additional production requirement
Context	Exact control records and payload references	Learned extraction, useful relevance selection, accounting for model tokens
Governance	Bound grants, expiry, scoped views	Authenticated identities, policy issuance, isolation, broader leakage analysis
Recovery	Checkpoint reconstruction and journal reconciliation	Storage durable across crashes and external service guarantees
Coordination	Declared footprints, real threaded jobs, checked deltas	Reliable footprint discovery, distributed scheduling, isolation of external effects
Migration/reuse	Two envelope versions; procedure templates for one action	Real runtime adapters and transfer evaluation
Retention	Managed payload and descendant deletion	Backup policy, encryption, exported copies, metadata classification

Table 4: Scope of the reference implementation and complementary requirements for production deployment.

journal and admits a linked receipt. Logical operation identities prevent repeated dispatch from being interpreted as a fresh effect, while compensation is represented by a separately authorized operation.

The host process, admission interface, and principal labels are trusted. Hash linkage detects inconsistent recorded transitions but does not authenticate a log against wholesale replacement. These assumptions define the reference model's enforcement boundary; production identity, durable storage, isolation, and service integration are complementary requirements summarized in Table 4.

7.2 Coordination and transfer mechanisms

A delegated branch receives a copied view, a declared read/write footprint, and a resource reservation. Independent tasks execute through a thread pool and return deltas for admission. Merge checks the stored branch contract, input revisions, permitted writes, and reserved resources before admitting a batch. The reference callbacks are trusted computations without effects; the scheduler consequently tests local concurrency and conflict handling without assuming isolation of arbitrary external effects.

Handoff validates support for the record kinds required by a destination view. Migration provides a reversible mapping between two defined envelope versions, preserving the underlying execution semantics. Procedure extraction derives a parameterized update template from a verified case containing a single operation. Instantiation requires a new target version and grant. These realizations expose transfer and derivation contracts in a form that can be exercised by tests; migration specific to a framework and generalized procedure learning remain outside the implementation.

7.3 Illustrative procurement execution

The procurement example connects these mechanisms within one execution. A supplier quote is admitted with its source identity, while a proposed recommendation is recorded as an inference. Repeated document payloads can share storage without merging provenance. A derived ranking binds the quote revision, and a compact view retains the records relevant to the decision while resolving detailed terms through the payload store.

Two workers then receive subgoals for checking suppliers, restricted views, and separate resource reservations. Their independent analyses execute concurrently and return artifacts bound to evidence. The parent admits compatible results, settles resources, and preserves disagreements

for resolution. If a quote changes, the associated ranking and proposal become stale while unrelated evidence remains current. The same dependency relation supports both selective recomputation and admissibility checks at merge.

The case of a lost acknowledgement from Section 1 exercises the same state across dispatch, restoration, reconciliation, and closure. Its tested sequence and observed effect count are reported in Section 7.5.

This walkthrough links the implemented mechanisms; it is not an autonomous procurement deployment. The validation below exercises individual operators and six composition scenarios.

7.4 Validation method

The reference evaluation checks operator behavior on positive and adversarial cases and the preservation of execution relationships across selected compositions. Table 5 gives the method counts and family coverage. These are authored assertions against the implementation, rather than an exhaustive analysis of the state space or an independent verification.

Cases for individual operations cover canonical identity, dependency-closed projection, exact retrieval, reuse sensitive to dependencies, authorization, restoration, compensation, concurrent computation, resource settlement, schema transfer, and closure based on evidence. Adversarial inputs exercise unsupported values, dependency cycles, inaccessible or corrupted payloads, stale caches, revoked and expired grants, duplicate dispatch, changed branch bases, altered branch contracts, unsupported destination semantics, incomplete event records, and unjustified closure.

The composition scenarios examine compaction followed by scoped rehydration; forked execution followed by parallel admission; interruption followed by restoration, reconciliation, and closure; revocation followed by invalidation and attempted cache reuse; transfer and reconstruction across supported representations; and deletion followed by attempted retrieval or replay. These scenarios instantiate the boundary conditions in Section 5. Test identifiers, outcomes, interpreter version, and source hashes are recorded by the evidence runner described in Appendix C.

7.5 Contract and composition checks

All methods pass (Table 5); multiple assertions within a method are not counted as additional test cases.

Test group	Methods	Passed	Coverage
Checks of individual operations	49	49	Positive and adversarial cases across 19 families
Composition scenarios	6	6	State relationships across operations
Total	55	55	Zero failures or errors

Table 5: Reference evaluation results. Counts are generated from the executable test suite. These are functional checks of the local model, not benchmark episodes for interactive agents.

Two scenarios illustrate the distinction between an operation's local behavior and its role in a longer execution. The concurrency test uses a synchronization barrier requiring two worker threads to be active, while a separate case assigns conflicting footprints to different batches. This establishes concurrent execution and scheduling sensitive to conflicts under the declared footprints. Admission subsequently checks the base revisions and delegated writes, connecting the scheduler's assumptions to the parent's acceptance of returned state.

In the scenario with a lost acknowledgement, the service records an effect while the corresponding attempt remains uncertain. The test compacts the case view, restores a checkpoint, rejects premature closure, reconciles the service journal, and then accepts completion. The service

contains one realized effect throughout. The preserved relationship is not merely a remembered description of the task: it is the binding among the logical operation, uncertain attempt, service evidence, and unresolved obligation. This relationship determines which continuation is admissible after interruption.

These results demonstrate executable consistency for the specified cases. They do not estimate production failure rates, establish distributed guarantees, or measure changes in task success of autonomous agents.

7.6 Representation size

A separate diagnostic measures the serialized size of two views over the same quote fixture. The complete view occupies 4565 UTF-8 bytes, whereas its compact form containing references occupies 947 bytes. The externalized payload occupies 3600 bytes and remains in the store. Both representations retain the selected control records and their bindings; the difference is the inclusion of inline payload.

The diagnostic isolates one mechanism by which structure can reduce the size of a decision view. It is not a measure of net storage reduction or language model token efficiency. Total execution cost depends on the construction of the case, retrieval frequency, cache behavior, repeated context use, and any effect on task performance. The retained payload is part of that accounting. These quantities motivate the comparative design in Section 6.

8 Empirical Effects of Structural Operations

We evaluate three questions at different levels. First, when the input and execution requirements are held fixed, what changes when a structural strategy is enabled? Second, can equally typed storage implementations recover the same intended state after process interruption, and at what cost? Third, is the agent controlled by a model ready for a larger comparison of task completion? We address these questions with controlled mechanism workloads, durable storage trials, and an AppWorld readiness pilot, respectively (Figure 3). Their experimental units and outcomes remain separate throughout the analysis.

8.1 Study design and comparisons

Comparison and controls (Figure 3A). The mechanism study uses generated, scripted workloads, not language model episodes. Each workload supplies a case, an operation sequence, and an expected outcome. Every input is executed with a *baseline* safe fallback and an *enabled* strategy. Both retain the same typed case core, admitted information, authority policy, executor, resource cap, and outcome requirements. Disabling a strategy leaves the safeguards intact. Thus, the comparison measures a strategy's effect within the reference runtime, not the effect of replacing an unstructured transcript with typed state.

A concrete paired comparison (Figure 3B). For compaction, both configurations start from the same control records and payloads. The baseline renders payloads inline; the enabled strategy renders exact references and retains the backing payload. We measure delivered bytes and check required values and bindings against the same expected result. This isolated workload requests no detailed payload; subsequent access is measured in the compaction and retrieval comparison. Caching compares repeated computation with reuse checked against revisions. Protection operations compare alternatives enforcing the same policy, evaluating preservation against a baseline that is already correct.

Workloads and regimes (Figure 3C). We use nineteen workloads for single families and seven selected workloads combining two families. The latter execute four configurations: neither

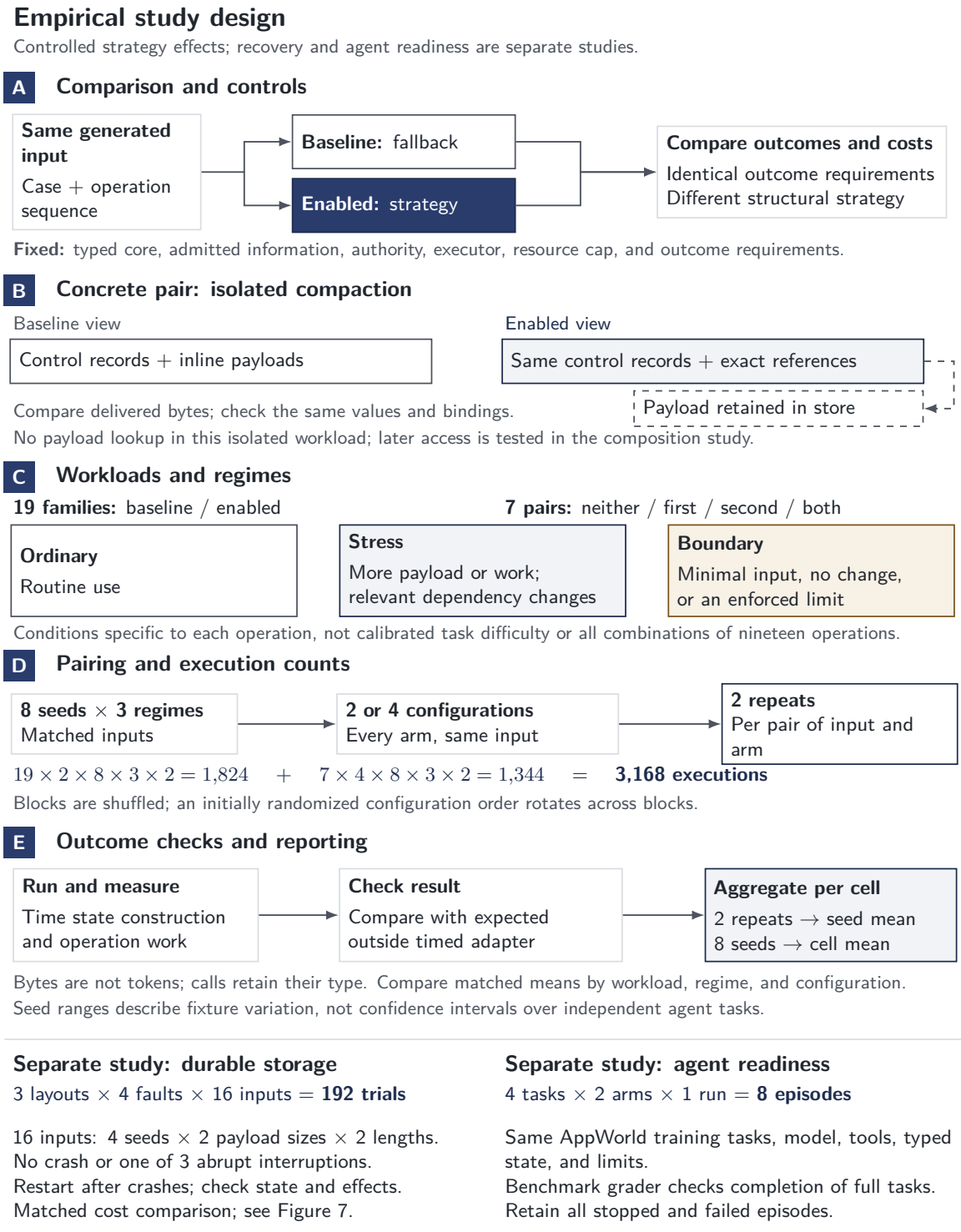


Figure 3: The empirical protocol. Panels A–E map to the five parts of Section 8.1: fixed controls, a concrete compaction pair, workload conditions, pairing and execution counts, and independent outcome checks followed by aggregation. Panel A depicts a comparison for a single family; workloads combining two families use all four switch combinations listed in C. Panel B separates the delivered view from retained backing payload. Counts include repeated executions, not independent agent tasks. The two studies at the bottom use different experimental units and are reported separately.

strategy, the first only, the second only, and both. This tests each operation with and without

its partner, not all combinations of nineteen families. Each workload has three regimes: *ordinary* exercises routine use; *stress* increases relevant payload or repeated work and, where applicable, changes a dependency; *boundary* exercises minimal inputs, no change, or limits such as an exhausted resource cap. These are conditions specific to each operation, not calibrated levels of general task difficulty.

Pairing and execution counts (Figure 3D). Eight generator seeds define reproducible inputs for each workload and regime. Configurations share an input within each matched block, and each combination of input and configuration runs twice. Blocks are shuffled; an initially randomized configuration order rotates across blocks to reduce order effects. Workloads for single families contribute $19 \times 2 \times 8 \times 3 \times 2 = 1{,}824$ executions; workloads combining two families contribute $7 \times 4 \times 8 \times 3 \times 2 = 1{,}344$, totaling 3,168. One execution applies one configuration to one generated input once. In workloads for single families, equivalent add-one and leave-one-out labels denote the same two configurations and are not counted again.

Outcome checks and reporting (Figure 3E). Expected outcomes are held outside the timed adapter and checked against its observed result; the adapter cannot grade itself. All 3,168 scheduled executions completed and passed their specified checks. We average the two repeats within each seed, then the eight values at the seed level, separately by workload, regime, and configuration. Reductions compare matched enabled and baseline means. Figure 4 also shows nonzero ranges of seed means: these describe fixture variation, not confidence intervals over agent tasks. Several protection workloads repeat the same semantic cases across seeds.

Bytes measure UTF-8 serialization or delivered payload, not model tokens; backing payload is accounted for separately. Calls count relevant store, compute, worker, or service requests. Timing includes state construction and operation overhead, excluding fixture generation, grading, and ledger writing. Appendix D reports all families and regimes, including unchanged outcomes and added cost. We present the operation results first, followed by recovery and model readiness.

8.2 Context volume and repeated computation

Enabling compaction, projection, and retrieval reduced delivered context in the stress workloads (Figure 4A). The figure reports each strategy against its named fallback; Table 10 retains the unrounded byte means. These are distinct paired workloads, not additive savings. Selective retrieval also reduced store reads from ten to one. In each comparison, the external payload of 18,000 bytes remained stored: the result concerns the consumer view, not net storage elimination.

Structure also supported reductions in repeated work: caching checked against revisions, selective invalidation, and handling of stale branches at merge each required fewer calls than its fallback (Figure 4B). These workloads supply the relevant dependency relationships and use scripted computations; they measure reuse and selective refresh under those relationships, not the accuracy of learning dependencies from an agent transcript.

Canonicalization produced a complementary representational result: equivalent content received a single canonical encoding rather than several distinct encodings (Table 6). The result held in all three regimes across twelve supplied encodings, without changing serialized content size or semantic equivalence. Canonical identity is therefore an observed consistency property here, rather than a compression result.

8.3 Workload dependence and composition

The magnitude of the improvements depended on the workload (Figure 5). Compaction's byte reduction increased with payload size. Projection and selective retrieval produced no byte reduction in the boundary regime. Selective invalidation and merge reduced computation when the stress workload changed a dependency, but left call counts unchanged without that change.

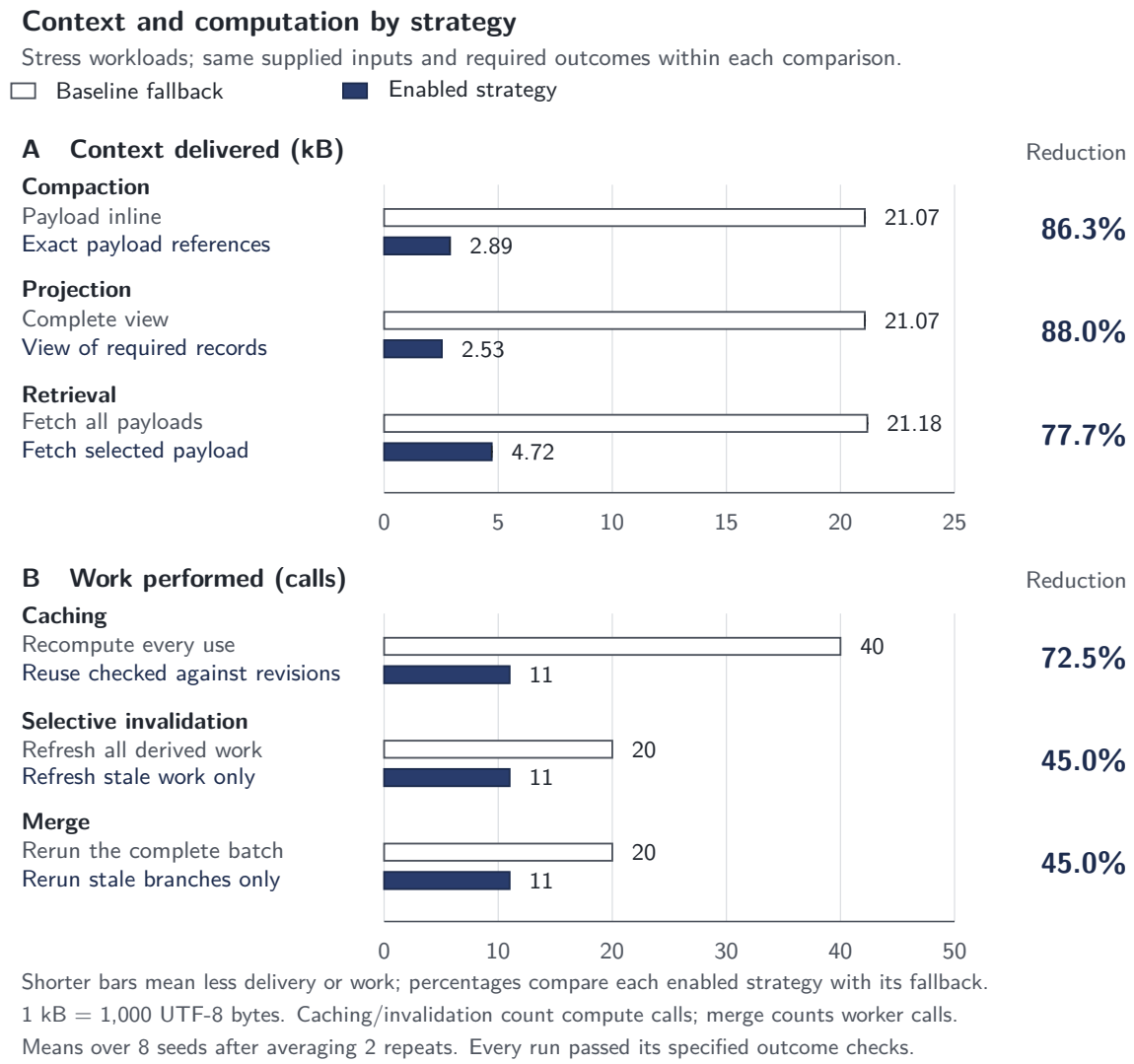


Figure 4: Named strategy comparisons under stress. Each pair labels the baseline fallback (open bar) and the enabled strategy (filled bar), making the source of the reduction explicit. (A) Delivered context in decimal UTF-8 kilobytes. (B) Compute calls for caching and invalidation, and worker calls for merge. Shorter bars indicate less delivery or work; percentages give the reduction from the corresponding fallback. Means summarize eight seeds after averaging two repeats; visible whiskers show their observed range, not confidence intervals. All specified outcome checks passed.

Caching reduced compute calls in all three regimes. These contrasts identify the supplied conditions under which gains occurred rather than treating every enabled operation as uniformly beneficial.

The compaction and retrieval workload asks whether compaction's isolated size reduction survives later access to the payload. The finding is that *retrieval policy determines whether the delivery saving persists* (Figure 6). Under stress, compaction followed by eager reconstruction increased total delivery and introduced store reads. Compaction followed by selective retrieval retained a substantial saving and required only one read. Retrieval alone had no effect because the payload remained inline. Figure 6 reports all four configurations, including the additional delivery incurred by eager reconstruction.

The comparison therefore measures total delivery through the operation chain, not merely the size immediately after compaction. The benefit is also conditional on workload: at the boundary

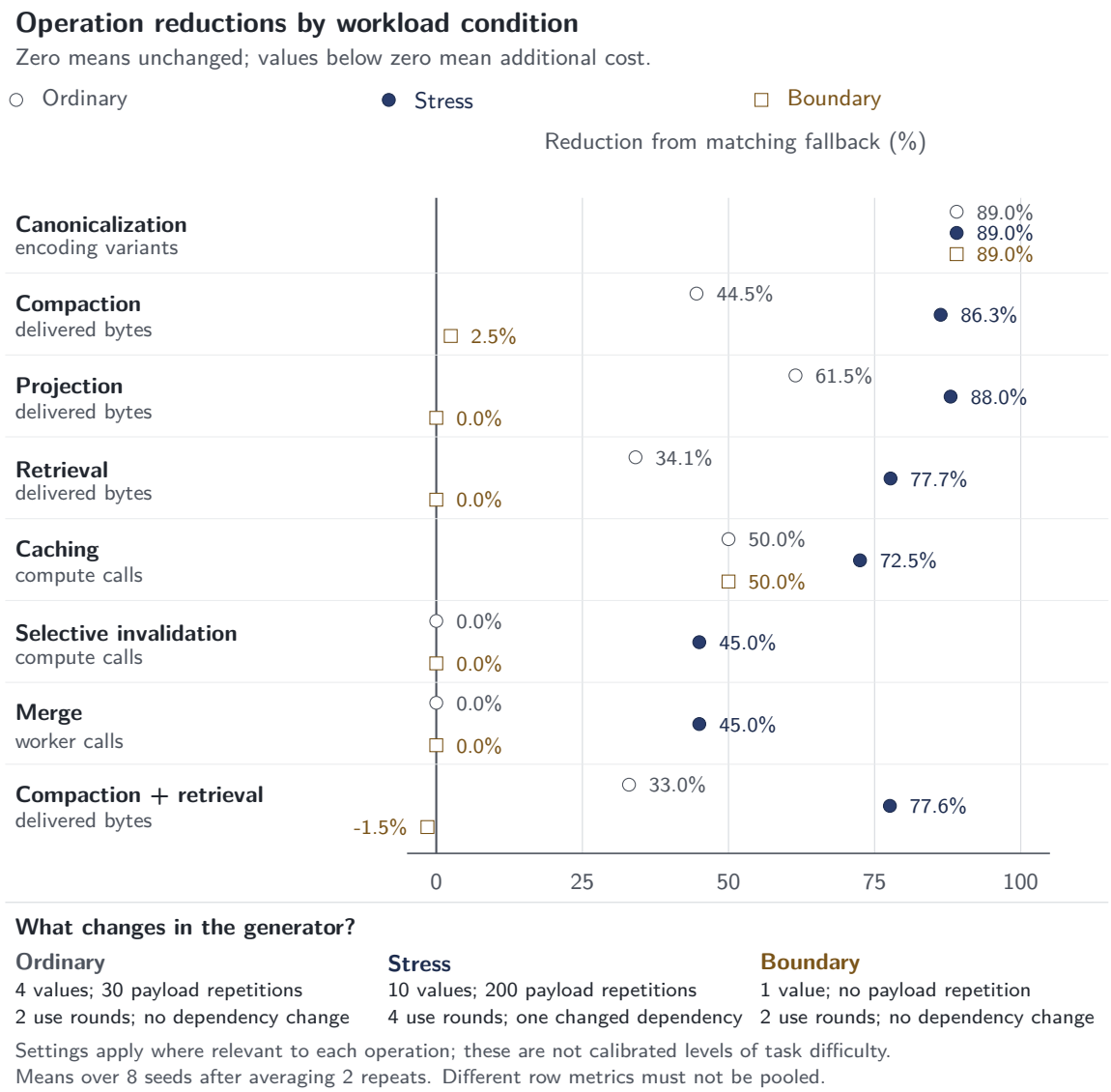


Figure 5: Workload conditions and measured reductions. Each marker shows $100(1 - \bar{x}_{\text{enabled}}/\bar{x}_{\text{baseline}})$ for the metric beneath its operation. The vertical offsets distinguish conditions, not another quantitative variable. The fixture settings below the axis explain what ordinary, stress, and boundary mean; their relevance varies by operation. The boundary compaction and retrieval cost lies to the left of zero. The final row compares both strategies with neither; other rows compare one enabled strategy with its safe fallback. Means summarize eight seeds after averaging two repeats. Metrics must not be pooled into one performance score.

with small inputs, both strategies increased delivery from 727 to 738 bytes and introduced one read. These results identify a useful composition and its boundary cost, rather than assuming that two locally useful operations always produce a joint improvement.

Two further comparisons reduced avoidable requests without altering realized outcomes (Table 6). Reconciliation using a journal replaced an idempotent retry write with a journal query; both strategies retained exactly one realized effect. Resource prechecking eliminated rejected requests in the stress regime, with identical spent work units. Neither result implies that all requests or all costs disappeared: reconciliation changed the request type, while resource control avoided requests that the same underlying cap would reject.

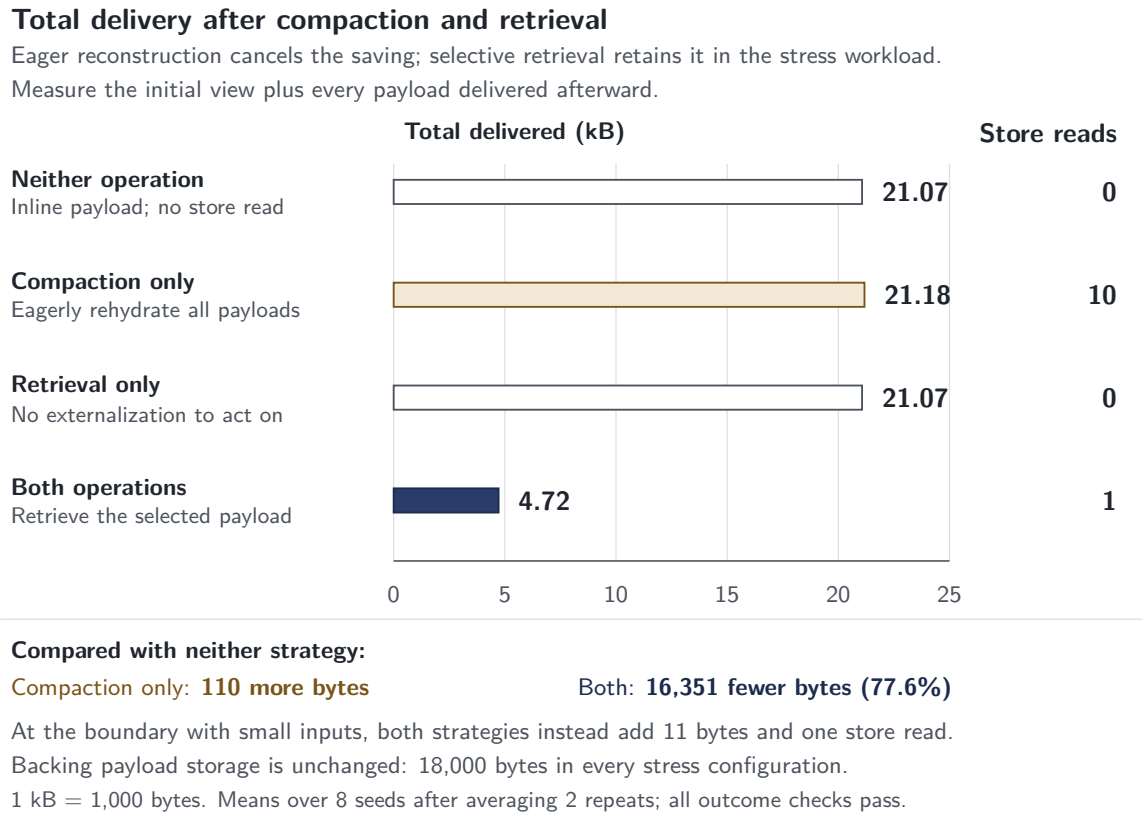


Figure 6: The compaction saving depends on what is retrieved afterward. Bars show total delivered context for all four configurations of the same stress workload; the adjacent column gives store reads. Eager retrieval after compaction reconstructs all payloads and erases the saving, whereas selective retrieval retains it. Thus, "compaction only" includes downstream eager access here and differs from the isolated compaction measurement without retrieval in Figure 4. Values are means over eight seeds after averaging two repeats. Backing storage and required outcome checks are unchanged.

8.4 Correctness preservation and recovery

Both arms passed all 48 runs in each of the four protection families: authorization, scoped disclosure, completion verification, and retention. The target is preservation of correct decisions, not improvement beyond a baseline that is already correct. Authorization covered valid, revoked, and expired grants, changed proposals, and wrong principals; disclosure excluded forbidden records and derivatives; completion distinguished success from missing effects, missing acknowledgements, and open branches; retention removed covered payloads while preserving unrelated material. Each run checked all conditions in its family. Repeated semantic fixtures establish behavior under these contracts, not independent security challenges or a deployment failure probability.

The separate durable storage study extends recovery beyond reconstruction in memory. It compares a unified indexed table, separate module tables in one transactional database, and separate module databases with a durable coordinator. Each layout receives the same combinations of four seeds, two payload sizes (64 and 4,096 bytes), two workflow lengths (one and eight cases), and four fault conditions: $3 \times 4 \times 2 \times 2 \times 4 = 192$ trials. One trial executes one workflow under one layout and fault condition. Of these, 144 use abrupt termination of a child process after intent persistence, after the service effect, or after the third record write; the remaining 48 run without interruption. A new process recovers the interrupted work. An independent verifier compares all six record roles with the persistent service state.

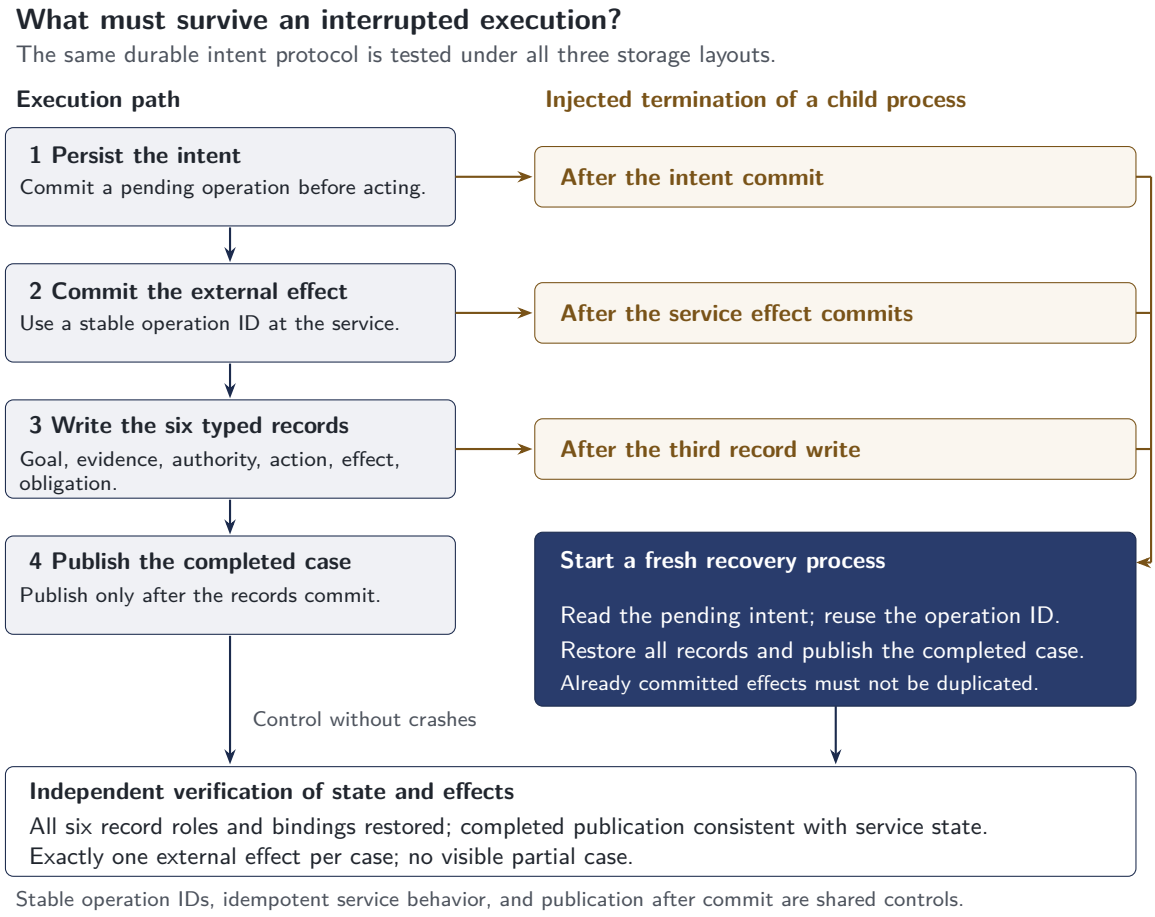


Figure 7: The durable storage interruption experiment. The left path shows normal execution; each gold branch terminates the child process at one specified point and starts recovery in a fresh process. The third branch interrupts record construction after record three, not after all six records are written. Both normal and recovered workflows face the same independent checks. The service, operation identities, typed semantics, and durable intent protocol are common controls across the three storage layouts. All 192 trials, including 144 interrupted and 48 trials without crashes, completed successfully.

All 192 trials completed successfully across the three layouts, with exactly one external effect per case and no visible partial cases.

Storage costs differed despite these identical outcomes. We report medians of 64 differences for matched inputs per comparator, matching on seed, payload size, workflow length, and fault condition. Relative to module tables in one database, unified storage used 36,864 fewer file bytes and took 0.56 ms less time; commit counts, service requests, attempts to write records, replayed intents, and intent payload bytes were identical in every matched block. Relative to separate module databases, unified storage used 57,344 fewer file bytes and 31.5 fewer commits, and took 16.67 ms less time; median differences in the other four metrics were zero. These descriptive local measurements characterize physical storage and transaction boundaries under equal typed semantics, not an advantage over alternative semantic representations or latency of tasks executed using a model.

8.5 Overheads and readiness for execution using a model

Several operations introduced overhead in the measured workloads. Table 6 reports these costs alongside the additional benefits in representation and requests. Checkpointing, handoff, and procedure extraction and reuse increased serialized transfer or state size. Local parallel execution

took longer than sequential execution for the small worker workload without effects; checked replay took longer than a baseline journal scan. These costs caution against extrapolating concurrency or replay benefits from operation availability alone.

Operation	Metric	Baseline	Enabled	Difference
Canonicalization	Distinct encodings	9.125	1	-8.125
Reconciliation	Retry writes	1	0	-1
Reconciliation	Journal queries	0	1	+1
Resource control	Rejected requests	4.625	0	-4.625
Checkpointing	Transfer bytes	2,491	3,717	+1,226
Handoff	Transfer bytes	21,108.375	21,142.375	+34
Procedure reuse	State bytes	37,891.625	38,915.625	+1,024
Parallel execution	Local ms	3.671	4.851	+1.18
Checked replay	Local ms	2.383	3.046	+0.663

Table 6: Additional effects and overheads under stress. Values are means over eight seeds after averaging two repeats; difference is enabled minus baseline. Each arm passed all 16 specified outcome checks. Byte counts are serialized UTF-8 quantities, not tokens. Local times include state construction and operation work. All regimes and nineteen families remain in Table 10.

Finally, an AppWorld training pilot using a language model [26] compared compaction enabled and disabled on four development tasks with one run per task and arm. Both used the same typed representation, tool interface, model snapshot (`gpt-5.4-mini-2026-03-17`), low reasoning effort, and limits of 48 decisions, 32,000 input tokens per decision, and 2,048 output tokens per response. Neither arm fully completed a task. Table 7 pairs each task's grader checks and stop reason across arms; these checks specific to each task are not pooled as an accuracy score. Compaction externalized older payloads on three tasks, but the prespecified readiness condition of at least two baseline successes was not met, and the study did not expand. Appendix D retains the remaining gate condition and episode accounting.

Task	Baseline checks	Compact checks	Baseline stop	Compact stop
`302c169_1`	3/8	2/8	Finish	Finish
`b0a8eae_3`	4/5	0/5	Finish	Context limit
`afc0fce_2`	6/9	3/9	Abstention	Incomplete model response
`ce359b5_1`	2/8	2/8	Decision limit	Abstention

Table 7: Complete AppWorld training pilot on four tasks, one episode per task and arm. Full task success was 0/4 in each arm. A finish signal is the agent's stop decision, not independently verified success. Check denominators are specific to each task and must not be pooled. All stopped and failed episodes are included.

9 Discussion

9.1 Scope of unification

Unification concerns a shared semantic interface, not uniform algorithms or one physical database. Capabilities retain their own policies and enforcement mechanisms while deriving their inputs and conditions from common execution relationships.

This perspective identifies potential benefits beyond a reduction in context length. A source identity created for retrieval can also support invalidation, audit, and merge. An obligation

retained for recovery can also constrain completion and guide a handoff. Figure 8 illustrates this reuse through a change in source revision. A view, cached analysis, proposal, and worker result each refer to the prior revision. The retained dependency identifies where currentness must be checked, while the appropriate response remains specific to each capability: refresh the view, invalidate the cache, revalidate the proposal, or check the branch at merge. A source update does not itself issue or revoke permission; authorization still requires the current proposal and policy checks. The central empirical question is whether sharing these relationships improves continuity or reduces integration cost relative to carefully engineered alternatives with explicit state.

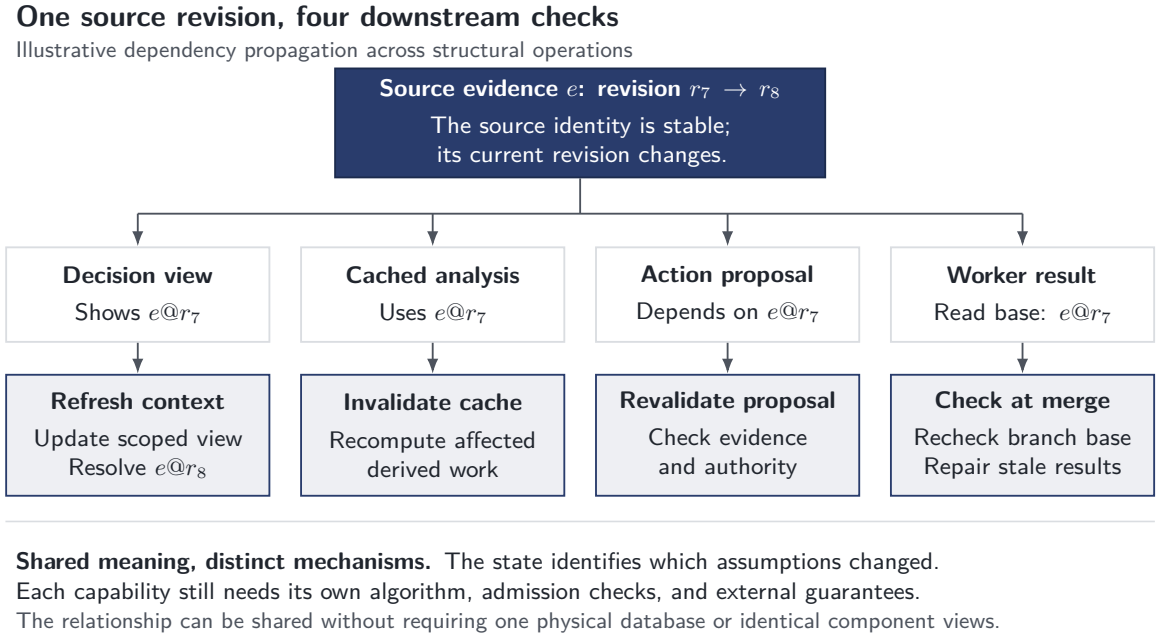


Figure 8: One recorded dependency supports different operational checks. In this illustrative scenario, four consumers depend on source revision r_7, which is superseded by r_8. The same source identity and revision binding locates the affected consumers, but each capability applies its own response. The diagram explains the proposed role of structured state across operations; it is not an additional experiment, a measured reduction in integration cost, or a claim that a schema alone enforces the checks.

Record types specific to the domain and views specific to each component remain possible. Interoperability requires agreement on identity, validity, authority, and completion beyond uniform field names.

9.2 Determinism, evidence, and enforcement

Structure permits deterministic treatment of selected parts of an otherwise stochastic system. Canonical encodings, lifecycle reducers, reference checks, and replay can have reproducible behavior under fixed implementations and inputs. Model proposals and environmental responses remain potentially nondeterministic. The architectural objective is to constrain their consequences through explicit state transitions and persistent uncertainty, rather than to require identical future model responses.

Three boundaries limit what this control can establish. First, structural validity does not guarantee factual accuracy: a typed observation may be incorrect, and an extracted goal may omit a requirement. Second, an internally consistent record does not authenticate its source or authorize its use. Third, an authorized action does not prove that its intended effect occurred.

Admission, identity services, policy enforcement, and effect evaluation therefore remain essential components of the runtime.

Research on prompt injection illustrates the importance of distinguishing external content from instructions and authority [11]. A typed case makes that separation explicit, but its effectiveness depends on the gate, tool isolation, and treatment of derived content. The reference implementation assumes trusted admission and host execution; it does not evaluate adversarial compromise of those components. Similarly, a deterministic completion check can be incomplete if its predicate omits a required submission or collateral effect. Evaluation must examine the coverage of the contracts as well as their implementation.

9.3 Costs and environmental constraints

A structured case introduces costs in schema design, interpretation, storage, version management, retrieval, and adapter maintenance. These costs may dominate on short tasks or workloads with little reusable state. Excessively detailed records can also reproduce the growth problem of a transcript in a more elaborate encoding. A practical implementation must determine which state remains relevant to future decisions and which evidence can be retained behind resolvable references.

The potential amortization of these costs is across operations. A dependency recorded once may serve compaction, invalidation, delegation, and audit. Section 8 measures concrete reductions in repeated work and context delivery, while also showing that some gains disappear in small or unchanged workloads. The compaction and retrieval comparison further shows that downstream reconstruction policy determines whether an intermediate size reduction persists. These results motivate, but do not replace, the stronger comparisons with structured prose and explicit state in Section 6, which measure costs of full episodes and isolate the effect of a common representation from improvements in surrounding tools.

External effects impose an additional constraint. Atomicity, idempotency, settlement, and reversibility are properties of service mechanisms, not consequences of a case schema. A merged state cannot reverse an email already sent by a rejected branch. A restored attempt cannot establish the current state of an unavailable service. Where an effect cannot be resolved, the case must preserve that uncertainty and restrict subsequent claims of completion. The reference service models versioned values and reconciliation using a journal; distributed failures and irreversible physical effects require separate implementations and evaluations.

9.4 Retention, portability, and reuse

Retention, portability, and reuse impose competing demands on the same dependencies. Retaining evidence supports audit and transfer but can conflict with deletion requirements; exporting a derived procedure creates another artifact whose source availability and permission must be tracked. A lifecycle policy must therefore specify which original and derived artifacts remain usable after a source is retired, and which reconstruction claims must be withdrawn.

The implementation boundaries are indexed in Appendix A: managed deletion, a migration between two envelopes, and a template for a single action do not establish control of exported copies, interoperability among deployed frameworks, or task generalization. Evaluating these capabilities together requires tracking destination reference access, fresh authority, failed transfers, and the cost of revalidating reusable artifacts.

9.5 Limitations of the present evidence

The formal results are conditional arguments about declared relationships and transition contracts, rather than proofs of the implementation checked by machine. The reference tests and controlled workloads exercise authored cases and selected failures; they do not enumerate the state space.

Concurrency uses trusted local workers, dependencies and relevance roots are supplied, and recovery retains access to a simulated service journal. Byte reductions measure serialized or delivered data, not savings in model tokens. Both mechanism arms already use the typed case core, and the durable study varies physical storage under equal semantics. Neither experiment isolates the advantage of the proposed representation over a matched alternative semantic interface.

The pilot using a language model is limited to four development tasks and one run per arm. Neither arm completed a task, and the readiness gate prevented expansion. Its lower unconditional cost in the compaction arm is not evidence of better efficiency at equivalent task success. The zero violations in controlled protection cases likewise do not establish a population failure rate, a security guarantee, or statistical noninferiority.

The evidence supports representational expressiveness, contract realization, and controlled operation effects. Claims about tasks and representations remain subject to the matched comparisons in Section 6.

10 Conclusion

We introduced the typed execution case as a versioned representation of goals, evidence, authority, actions, effects, and unresolved obligations. Nineteen structural operation families share this representation through explicit contracts; the reference implementation satisfies 55 contract tests for individual operations and their compositions.

The controlled study recorded 3,168 executions satisfying their specified outcome checks, with reductions in delivered context and repeated work that varied by workload. Composition changed those benefits: eager retrieval erased compaction's delivery saving, whereas selective retrieval retained it. Workloads with small inputs and coordination workloads exposed additional costs. All 192 durable storage trials recovered or completed correctly, including 144 abrupt process interruptions.

These findings establish measurable benefits of individual strategies and preservation of the tested contracts within a common typed runtime. They do not isolate the benefit of sharing the representation itself: the mechanism comparisons retain the same typed core, and the agent pilot on four tasks achieved zero full successes in either arm. The central hypothesis therefore remains a distinct empirical question. Its decisive test is a matched comparison of shared execution state with equally capable, separately maintained state, measuring cost of reconstruction across operations and completion of full tasks under the same information, authority, and resource constraints.

References

[1] ContextAttest contributors. ContextAttest / Open Context: Policy-aware context infrastructure. Pre-alpha software repository; accessed September 7, 2026, 2026. URL https://github.com/doncazper/open-context.

[2] Jayanaka Dantanarayana, Savini Kashmira, Lingjia Tang, and Jason Mars. SIGIL: Compiling agent skills into typed harnesses. *arXiv preprint arXiv:2607.27309v2*, 2026. URL https://arxiv.org/abs/2607.27309v2.

[3] Kutluhan Erol, James Hendler, and Dana S. Nau. HTN planning: Complexity and expressivity. In *Proceedings of the National Conference on Artificial Intelligence*, pages 1123–1128, 1994.

[4] Richard E. Fikes and Nils J. Nilsson. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4):189–208, 1971.

[5] Martin Fowler. Event sourcing, 2005. URL https://martinfowler.com/eaaDev/EventSourcing.html.

[6] Samira Ghodratnama. Personalization and human context of LLM. In *Handbook of Human-Centered Artificial Intelligence*, pages 1–57. Springer, 2025.

[7] Samira Ghodratnama and Mehrdad Zakershahrak. Adapting llms for efficient, personalized information retrieval: Methods and implications. In *International conference on service-oriented computing*, pages 17–26. Springer, 2023.

[8] Samira Ghodratnama and Mehrdad Zakershahrak. Sumrecom: A personalized summarization approach by learning from users' feedback. *arXiv preprint arXiv:2408.07294*, 2024.

[9] Samira Ghodratnama, Mehrdad Zakershahrak, and Fariborz Sobhanmanesh. Adaptive summaries: A personalized concept-based summarization approach by learning from users' feedback. In *International conference on service-oriented computing*, pages 281–293. Springer, 2020.

[10] Samira Ghodratnama, Amin Beheshti, Mehrdad Zakershahrak, and Fariborz Sobhanmanesh. Intelligent narrative summaries: From indicative to informative summarization. *Big Data Research*, 26: 100257, 2021.

[11] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.

[12] Headroom contributors. Headroom: Context optimization for LLM applications. Software repository; accessed September 7, 2026, 2026. URL https://github.com/headroomlabs-ai/headroom.

[13] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12 (10):576–580, 1969. doi: 10.1145/363235.363259.

[14] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.

[15] Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. ACON: Optimizing context compression for long-horizon LLM agents. *arXiv preprint arXiv:2510.00615v3*, 2026. URL https://arxiv.org/abs/2510.00615. ICML 2026.

[16] LangChain. LangGraph persistence. Official documentation; accessed September 7, 2026, 2026. URL https://docs.langchain.com/oss/python/langgraph/persistence.

[17] Hangfei Lin. Architecting efficient context-aware multi-agent framework for production. Engineering article; accessed September 7, 2026, 2025. URL https://developers.googleblog.com/architecting-efficient-context-aware-multi-agent-framework-for-production/.

[18] Yanming Liu, Xinyue Peng, Jiannan Cao, Xinyi Wang, Songhang Deng, Jintao Chen, Jianwei Yin, and Xuhong Zhang. ToolGate: Contract-grounded and verified tool execution for LLMs. In *Findings of the Association for Computational Linguistics: ACL 2026*, 2026. doi: 10.18653/v1/2026.findings-acl.470. URL https://aclanthology.org/2026.findings-acl.470/.

[19] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1160–1183, 2025. doi: 10.18653/v1/2025.findings-naacl.65. URL https://aclanthology.org/2025.findings-naacl.65/.

[20] Drew McDermott et al. PDDL: The planning domain definition language. Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control, 1998.

[21] Bardia Mohammadi, Nearchos Potamitis, Lars Klein, Akhil Arora, and Laurent Bindschaedler. Atomix: Timely, transactional tool use for reliable agentic workflows. *arXiv preprint arXiv:2602.14849v2*, 2026. URL https://arxiv.org/abs/2602.14849v2.

[22] Luc Moreau and Paolo Missier. PROV-DM: The PROV data model. W3c recommendation, World Wide Web Consortium, 2013. URL https://www.w3.org/TR/prov-dm/.

[23] Anand S. Rao and Michael P. Georgeff. BDI agents: From theory to practice. In *Proceedings of the First International Conference on Multiagent Systems*, pages 312–319, 1995.

[24] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2021.

[25] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999.

[26] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 16022–16076, 2024. doi: 10.18653/v1/2024.acl-long.850.

[27] Natalia Trukhina and Vadim Vashkelis. Compress the context, keep the commitments: A formal framework for verifiable LLM context compression. *arXiv preprint arXiv:2605.17304*, 2026. URL https://arxiv.org/abs/2605.17304.

[28] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

[29] Michael Wooldridge. *An Introduction to MultiAgent Systems*. Wiley, 2 edition, 2009.

[30] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[31] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[32] Mehrdad Zakershahrak. Interpretable by Design: Query-Specific Neural Modules for Explainable Reinforcement Learning. *arXiv preprint arXiv:2511.08749*, 2025. URL https://arxiv.org/abs/2511.08749.

[33] Mehrdad Zakershahrak. Human–ai interaction in llm. In *Handbook of Human-Centered Artificial Intelligence*, pages 449–489. Springer, 2026.

[34] Mehrdad Zakershahrak and Samira Ghodratnama. Explanation, debate, align: A weak-to-strong framework for language model generalization. *arXiv preprint arXiv:2409.07335*, 2024. URL https://arxiv.org/abs/2409.07335.

[35] Mehrdad Zakershahrak, Samira Ghodratnama, and Mohammadali Karfarma. LLMs in modern agentic systems: A systems survey. Companion manuscript and public research artifact, 2026. URL https://xpounder-com.github.io/llms-in-modern-agentic-systems/.

[36] Saber Zerhoudi, Jelena Mitrovic, and Michael Granitzer. The compaction cliff in long-running AI agent memory. *arXiv preprint arXiv:2608.22752*, 2026. URL https://arxiv.org/abs/2608.22752.

[37] Lvmin Zhang and Maneesh Agrawala. View-oriented conversation compiler for agent trace analysis. *arXiv preprint arXiv:2603.29678v3*, 2026. URL https://arxiv.org/abs/2603.29678.

[38] Kunlun Zhu, Hongyi Du, Zhaochen Hong, Xiaocheng Yang, Shuyi Guo, Zhe Wang, Zhenhailong Wang, Cheng Qian, Xiangru Tang, Heng Ji, and Jiaxuan You. MultiAgentBench: Evaluating the collaboration and competition of LLM agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025. doi: 10.18653/v1/2025.acl-long.421.

A Operation Index and Reference Semantics

Table 8 maps the named operation families in Section 4 to their reference entry points. The identifiers in this appendix are stable keys for the implementation and test records, not a second numbering scheme for the main exposition. Several families share state primitives or span multiple methods. The final column summarizes the implemented semantics and their scope.

Table 8: Reference implementation of the structural operation families.

ID	Reference entry points	Implemented semantics and scope
S01	`canonical`, `admit`	Supported values, identity, required fields, revisions, and dependency checks; caller supplies trusted admission and source classification.
S02	`compact`	Exact selected control records with externalized payload; infeasibility under the byte budget; no learned summarization.
S03	`closure`, `project`	Required dependency closure, currentness, recipient scope; relevance roots are supplied.
S04	`retrieve`, payload resolver	Access, managed availability, and content digest; no model evaluation of retrieval policy.
S05	`deduplicate`, `memoize`, `cached`	Exact duplicate matching and cache use bound to revisions; semantic equivalence is not learned.
S06	`diff`, `current`, `invalidated`	Differences at the record level and transitive invalidation; dependencies must be declared.
S07	`authorize`, `revoke`, `dispatch`	Principal, proposal revision, grant interval, and checks of target versions; no identity provider.
S08	`redact`, `project`	Record and dependency scope; no unrestricted information flow theorem.
S09	`checkpoint`, `restore`	Event consistency and exact reconstruction of internal state; persistence supplied externally.
S10	`reconcile`, `compensate`	Evidence from the service journal, attempt identity, and separately authorized compensation.
S11	`fork`	Scoped view, declared writes, base binding, and budget reservation.
S12	`schedule`, `run_parallel`	Declared conflict batching and concurrent trusted callbacks; no automatic discovery of effect footprints.
S13	`merge`, `cancel_branch`	Sealed local branch contract, rejection of stale bases, atomic model admission, and reservation settlement.
S14	`handoff`, `migrate`	Destination kind support and reversible envelope mapping; no production framework adapter.
S15	`replay`, `historical`	Reconstruction of recorded state and revision lookup; no external reexecution or model regeneration.
S16	`verify_close`	Current goal predicates, linked service receipts, uncertain attempts, and open branches.
S17	`extract_procedure`, `instantiate`	Verified source for a template with one action and fresh proposal creation; no carried grant or learned general policy.
S18	`retire`, payload deletion	Managed payload tombstones and descendant invalidation; exported copies and arbitrary control metadata excluded.
S19	`charge`, `monitor`, reservations	Nonnegative integer resource accounting, signal of repeated proposals, and explicit exhaustion; not actual provider billing.

B Test Coverage by Operation

The counts per family below are generated from named test methods. Each method may contain multiple assertions or subcases. The 6 composition tests are additional to these rows and are included in the overall total of 55 methods.

Operation	Positive methods	Adversarial methods	Passed/total
S01	1	3	4/4
S02	1	1	2/2
S03	1	1	2/2
S04	1	2	3/3
S05	2	1	3/3
S06	1	1	2/2
S07	1	2	3/3
S08	1	1	2/2
S09	1	1	2/2
S10	2	2	4/4
S11	1	1	2/2
S12	1	1	2/2
S13	1	2	3/3
S14	1	1	2/2
S15	1	1	2/2
S16	1	2	3/3
S17	1	1	2/2
S18	1	2	3/3
S19	2	1	3/3

C Reproducing the Reference Results

The accompanying source artifact contains the reference runtime, unit tests, evidence runner, manuscript sources, and generated result records. The implementation and tests use the Python standard library. From the artifact directory, the suite is executed with

```
python3 scripts/run_reference_checks.py
```

The runner generates `results/reference-results.json`, a textual test log, and the manuscript's result macros and coverage table. The result record includes each test identifier and outcome, the interpreter version, and SHA-256 hashes of the implementation, test suite, and runner. The suite can also be executed through Python's `unittest` discovery.

The checks require no language model calls or network services. The external environment is the local versioned service described in Section 7. The supplied manuscript compiles with pdfLaTeX and BibTeX using standard packages. The artifact's README distinguishes reference validation, completed empirical measurements, and prospective representation comparisons. Appendix D describes the recorded runs and the separate path for reproducing figures; neither compiling the manuscript nor generating its figures launches a model experiment.

D Complete Mechanism Results and Pilot Accounting

Table 10 reports all nineteen workloads for single families and all three regimes. The operation names abbreviate the broader families in Section 4; each measurement applies to the implemented strategy, not necessarily every suboperation in that family. In particular, canonicalization does not test learned admission; reconciliation does not evaluate general compensation; procedure reuse uses a template for a single action; and retention covers managed payloads only.

Table 10: Complete controlled mechanism measurements. Each cell gives baseline / enabled means over eight generator seeds after averaging two repeats. All 16 runs per arm and regime satisfied the specified outcome checks. Timing is local, including state construction and operation overhead; it is not agent latency.

Operation	Metric	Ordinary	Stress	Boundary
Canonicalization	Encodings	9.125 / 1	9.125 / 1	9.125 / 1
Compaction	View bytes	2,587 / 1,435	21,067.375 / 2,887.375	727 / 709
Projection	View bytes	2,587 / 997	21,067.375 / 2,528	727 / 727
Retrieval	Delivered bytes	2,631 / 1,734	21,177.375 / 4,716.375	738 / 738
Caching	Compute calls	8 / 4	40 / 11	2 / 1
Selective invalidation	Compute calls	4 / 4	20 / 11	1 / 1
Authorization	Correct (%)	100 / 100	100 / 100	100 / 100
Scoped disclosure	View bytes	165 / 165	165 / 165	165 / 165
Checkpointing	Transfer bytes	2,491 / 3,717	2,491 / 3,717	2,491 / 3,717
Reconciliation	Retry writes	1 / 0	1 / 0	1 / 0
Delegation	Transfer bytes	0 / 1,065	0 / 2,517.375	0 / 339
Parallel execution	Local ms	0.877 / 1.303	3.671 / 4.851	0.261 / 0.423
Merge	Worker calls	4 / 4	20 / 11	1 / 1
Handoff	Transfer bytes	2,628 / 2,662	21,108.375 / 21,142.375	768 / 802
Replay	Local ms	0.54 / 0.704	2.383 / 3.046	0.2 / 0.272
Completion verification	Correct (%)	100 / 100	100 / 100	100 / 100
Procedure reuse	State bytes	18,075 / 18,943	37,891.625 / 38,915.625	8,166 / 8,956
Retention	Retained payloads	1 / 1	1 / 1	1 / 1
Resource control	Rejected requests	2 / 0	4.625 / 0	1 / 0

The cost columns have different meanings. Disclosure and retention are constrained outcomes, not quantities to minimize regardless of correctness. Delegation introduces a child envelope where execution by the parent alone requires no transfer. Checkpointing and handoff count transfer envelopes; the procedure workload counts retained serialized state. Parallelism and replay report local wall time, including state construction and operation work. None of these quantities is a substitute for the full token cost of an agent episode or completion.

The seven workloads with four configurations are compaction and retrieval, compaction and handoff, delegation and projection, parallelism and merge, checkpointing and reconciliation, invalidation and caching, and retention and replay. Their configurations contribute 1,344 of the 3,168 executions; the workloads for single families contribute 1,824. All recorded configurations are retained in the source artifact. The main text examines compaction and retrieval because its four cells expose the difference between reducing an intermediate view and reducing total subsequent delivery. It does not summarize the other pairs as universally beneficial interactions.

D.1 Model readiness pilot

Table 7 in the main text reports all four paired tasks, their grader checks specific to each task, and their stop reasons. Table 11 provides the corresponding accounting without conditioning on successful or normally terminated episodes.

The prespecified readiness gate required at least two baseline task successes and actual compaction exposure on at least two tasks. The observed values were zero and three, respectively, so the gate failed. All eight scheduled episodes were retained, including runs limited by context or decision count, runs with abstention, and runs with incomplete responses. Because stopping patterns and

achieved task progress differed, the unconditional cost and time totals do not establish improved cost per successfully completed task.

Metric	Baseline	Compaction
Fully successful tasks	0/4	0/4
False completion claims	2	1
Tasks with payload externalization	0/4	3/4
Model calls	110	88
Total cost of model usage (USD)	1.0981	0.7947
Mean total episode time (s)	141.54	101.74

Table 11: Pilot accounting across all four episodes per arm. Costs use the frozen price schedule and exclude earlier pilot attempts. Episode stopping and achieved progress differed, so lower unconditional cost or time is not improved cost per successfully completed task.

D.2 Recorded evidence and figure and table reproduction

The completed source runs have the following identifiers:

Mechanisms: `20260908T205855.254605Z-3a2d62c761`
Durable storage: `20260908T210507.958060Z-durable-v1`
Model readiness: `20260908T205652.299256Z-dbe53d4e37`

Configuration, schedule, raw observations, and source snapshots remain separate from the earlier reference checks and earlier pilot attempts. No held-out benchmark partition is used in these results.

The standard library evidence generator validates mechanism and durability record hashes, paired fixture identities, seed/repetition coverage, and agreement with the reviewed stress means. It emits native TikZ figures, the complete mechanism table, the comparison tables in the main text, and pilot accounting. Two accompanying JSON records retain source hashes, measurements at the seed level, matched durability differences, and the values behind each new table. The figures compile as vector artwork using the manuscript's typography, without external rendering services or model calls. Reproduction instructions and the distinction between compiling supplied exhibits and recomputing them from local raw evidence are included with the source package.